

ChatGPT ▾

CS 441 Program 2 Spr 2026.pdf
PDF

explain this assignment elaborately

Alright — this is a **big, important assignment**, and I'll break it down in a way that actually makes sense step-by-step (not just repeating the PDF). Think of this as your **mental model + implementation roadmap**.

🧠 What You're Building (Big Picture)

You are building a **mini-compiler front-end** in Racket.

👉 Input: a program written in a simple language

👉 Output: a structured tree (AST)

Example:

Input:

```
x := 10;  
WHILE x > 0 DO  
  x := x - 1;  
END
```



Output (AST):

```
'(program  
  (assign x 10)  
  (while (gt x 0)  
    (assign x (minus x 1))))
```



📌 This transformation is the **core goal** 📄 CS 441 Program 2 Spr 2026

🧩 The 2 Main Parts (VERY IMPORTANT)

Your program has **two stages**:

1. Scanner (Lexer)

👉 Converts raw text → tokens

Example:

```
"VAR x := 5;"
```



becomes:

```
'(VAR-TOK "x" ASSIGN-TOK 5 SEMI-TOK)
```



📌 Why?

Because parsing raw text is messy. Tokens make it structured.

2. Parser

👉 Converts tokens → AST using grammar

- Uses **recursive descent**
- Each function = one grammar rule

📌 Example:

- `parse-expression`
- `parse-term`
- `parse-factor`

🔄 Overall Flow

```
Raw Code
  ↓
strip-comments
  ↓
tokenize (Scanner)
  ↓
tokens list
  ↓
parse-program (Parser)
  ↓
AST (final output)
```

🔍 PART 1: Scanner (Lexical Analysis)

What it does:

Breaks text into meaningful pieces.

You must handle:

✅ 1. Tokens

- Keywords: `IF`, `WHILE`, `PRINT`
- Operators: `+`, `-`, `*`, `/`
- Assignment: `:=`
- Relational: `=`, `!=`, `>`, `>=`, `<`, `<=`
- Identifiers: `x`, `_abc-1`
- Numbers: `10`, `-5`, `+3.14`

✅ 2. Comments

```
/* comment */
```

You must:

- Remove them completely
- Handle weird cases like:

```
/* comment /* nested */ → stops at first */
```

📌 Comments do NOT appear in AST CS 441 Program 2 Spr 2026

✅ 3. Numbers (TRICKY)

Integers:

- Valid: 5, -10, +402
- Invalid: 05, -07

Floating Point:

- Valid: 0.5, -12.0
- Invalid: .5, 12.

📌 Must match strict grammar rules 📄 CS 441 Program 2 Spr 2026

⚠️ Unary vs Binary (VERY IMPORTANT)

```
x := -5;      → unary minus (part of number)
x := y - 5;   → binary minus (operator)
```



Rule:

- If before - is:
 - ID / number /)
 - binary
- Otherwise → unary

🔧 Helper Functions (Parser Tools)

These are your **core utilities**:

◆ peek(tokens)

- Look at next token
- Don't remove it

◆ consume(tokens, expected)

- Check if token matches expected
- If yes → remove it
- If no → ERROR

◆ strip-comments(string)

- Remove all /* ... */

🌳 PART 2: Parser (Recursive Descent)

This is where most logic happens.

🧱 Grammar → Functions

Each grammar rule = function

Example from page 5:

```
<Expression> ::= <Term> { ("+" | "-") <Term> }
```

Becomes:

```
parse-expression
```

⚙️ Core Parsing Functions

◆ parse-program

- Entry point
- Calls parse-stmt-list
- Ensures no tokens left

◆ parse-statement

Decides what to do based on first token:

Token	Function
IF	parse-if
WHILE	parse-while
PRINT	parse-print
ID	parse-assign

◆ parse-expression (LOW precedence)

Handles:

```
+ and -
```

Algorithm:

1. Parse a term
2. While next is + or -
3. Build tree

◆ parse-term (MEDIUM precedence)

Handles:

```
* and /
```

◆ parse-factor (HIGHEST precedence)

Handles:

- numbers
- identifiers
- parentheses

🔥 Precedence (SUPER IMPORTANT)

Highest → factor
 → term (* /)
 → expression (+ -)
Lowest



Example:

1 + 2 * 3



Correct AST:

(+ 1 (* 2 3))



NOT:

(* (+ 1 2) 3)



🔄 Left Associativity

10 - 5 - 2



Must become:

(- (- 10 5) 2)



NOT:

(- 10 (- 5 2))



📌 Achieved using loops in parser CS 441 Program 2 Spr 2026



Statements You Must Support

From grammar:

Assignment

x := expr;



If

```
IF condition THEN
  ...
ELSE
  ...
END
```



While

```
WHILE condition DO
  ...
END
```



Print

```
PRINT expr;
```



⚠ Errors You MUST Detect

Lexical Errors (scanner)

- `05`
- `.5`
- invalid symbols like `!`

Syntax Errors (parser)

Examples from page 6–7:

- Missing `END`
- Missing `ELSE`
- Wrong operator (`=` instead of `:=`)
- Missing `)`
- Invalid expression

📌 These must THROW errors, not silently fail CS 441 Program 2 Spr 2026

What You Should Test

Must pass:

```
x := 1 + 2 * 3 - 4 / 2;
```



Expected:

```
(assign x (- (+ 1 (* 2 3)) (/ 4 2)))
```



Deliverables (What You Submit)

1. Code

- .rkt files
-

2. AI Log

- Show:
 - mistakes
 - iterations
 - debugging
-

3. Video (3–5 min)

You must:

- Run your program
 - Explain logic
 - Discuss challenges
-

What Professors Care About (Grading)

From rubric (page 9):

High score =

- Correct AST
 - Handles edge cases
 - Clean recursive structure
 - Understands grammar deeply
 - Explains lexing vs parsing clearly
-

How to Approach This (BEST STRATEGY)

Step-by-step plan:

Step 1

Write:

strip-comments



Step 2

Write:

tokenize



Test thoroughly.

Step 3

Write helpers:

peek
consume

✔ Step 4

Build parser bottom-up:

- 1. parse-factor
- 2. parse-term
- 3. parse-expression
- 4. parse-statement
- 5. parse-program

✔ Step 5

Test small → big

🧠 Key Concept Summary

Concept	Meaning
Scanner	text → tokens
Parser	tokens → AST
AST	tree representation
CFG	grammar rules
Recursive descent	one function per rule
Precedence	operator priority
Associativity	order of grouping

🚀 If You Want

I can next:

- Walk you through **actual Racket code structure**
- Help you implement **tokenize step-by-step**
- Or debug your parser (very common for this assignment)

Just tell me 👍

Sources

first give the code for striping comments

Great — this is the **perfect place to start**, because everything else depends on clean input.

What strip-comments Must Do

From your assignment:

- Remove everything between:

```
/* ... */
```



- Comments **do NOT nest**
- Replace them with **space (optional but recommended)** to avoid merging tokens

Example:

```
<> racket
```



```
"X := 5; /* comment */ Y := 6;"
```

👉 becomes:

```
<> racket
```



```
"X := 5; Y := 6;"
```

Key Challenges

1. You must scan **character by character**
2. Detect:
 - start of comment → `/*`
 - end of comment → `*/`
3. Ignore everything inside
4. Resume normal scanning after `*/`

Clean Racket Implementation

Here's a **correct and simple recursive solution**:

```
<> racket
```



```
(define (strip-comments str)
  (define len (string-length str))

  (define (helper i in-comment?)
    (cond
      ;; End of string
      [(>= i len) ""]

      ;; If currently inside a comment
      [in-comment?
       (if (and (< (+ i 1) len)
              (char=? (string-ref str i) #\*)
              (char=? (string-ref str (+ i 1)) #\/))
           ;; Found end of comment "*/"
           (helper (+ i 2) #f)
           ;; Keep skipping characters
           (helper (+ i 1) #t))])

    ;; If NOT in comment
    [else
     (if (and (< (+ i 1) len)
              (char=? (string-ref str i) #\/))
```

How This Works (Step-by-Step)

It keeps track of:

```
<> racket
in-comment? ; true or false
```

Case 1: Not in comment

- If sees /* → enter comment mode
- Else → copy character

Case 2: In comment

- If sees */ → exit comment mode
- Else → skip everything

Test It

```
<> racket
(strip-comments "x := 5; /* comment */ y := 6;")
```

 Output:

```
<> racket
"x := 5; y := 6;"
```

Edge Case Test (VERY IMPORTANT)

</> racket

(strip-comments "/* comment /* nested */ still ignored */ x := 1;")

- 👉 Should behave like:
- First */ ends comment
 - Remaining */ is treated as normal text (and will cause syntax error later)

💡 Pro Tip (Important for Your Parser)

👉 We added " " when removing comments:

</> racket

(string-append " " ...)

Why?
Without it:

</> racket

"x:=5;/*comment*/y:=6;"

Would become:

</> racket

"x:=5;y:=6;"

✅ OK here, but in some cases tokens could merge incorrectly.

🚀 Next Step

Once this works, next you should implement: can you explain the given code line by line

👉 tokenize (scanner)

If you want's the guide strip-comments adding with tokenize, which is the next part after this 👉

</> racket

```
(define (strip-comments str)
  (define len (string-length str))

  (define (helper i in-comment?)
    (cond
      [(>= i len) ""]

      [in-comment?
       (if (and (< (+ i 1) len)
              (char=? (string-ref str i) #\*)
              (char=? (string-ref str (+ i 1)) #\/))
           (helper (+ i 2) #f)
           (helper (+ i 1) #t)))]

      [else
       (if (and (< (+ i 1) len)
              (char=? (string-ref str i) #\/)
              (char=? (string-ref str (+ i 1)) #\*))
           (string-append " " (helper (+ i 2) #t))
           (string-append
            (string (string-ref str i))
            (helper (+ i 1) #f)))))]))
```

Line-by-line explanation

1.

```
<> racket
```

```
(define (strip-comments str)
```

This defines a function named `strip-comments`.

- `define` creates the function
- `strip-comments` is the function name
- `str` is the input string, meaning the source code text

So this function takes the whole program as one string.

2.

```
<> racket
```

```
(define len (string-length str))
```

This stores the length of the input string in `len`.

Why do this?

Because later we keep checking where we are in the string, and we need to know when we reach the end.

Example:

If `str` is `"x := 5;"`, then `len` would be 7.

3.

```
<> racket
```

```
(define (helper i in-comment?)
```

This defines an inner helper function.

It uses two pieces of information:

- `i` = current position in the string
- `in-comment?` = whether we are currently inside a comment

So this helper keeps scanning one position at a time.

4.

```
<> racket
      (cond
```



`cond` is like a multi-branch `if`.

It checks conditions one by one until one is true.

5.

```
<> racket
      [(>= i len) ""]
```



This is the base case.

It means:

If `i` is greater than or equal to the length of the string, we are at the end.

Then return:

```
<> racket
      ""
```



which is the empty string.

Why?

Because there are no more characters left to process.

Second branch: when we are inside a comment

6.

```
<> racket
      [in-comment?
```



This branch runs if `in-comment?` is true.

That means we are currently somewhere between `/*` and `*/`.

So now the function should ignore characters until it finds the closing `*/`.

7.

```
<> racket
      (if (and (< (+ i 1) len)
```



This checks whether there is still another character after the current one.

Why?

Because to detect `*/`, we must look at **two characters**:

- current character
- next character

If we are at the last character, there is no next character to check.

8.

```
<> racket
      (char=? (string-ref str i) #\*)
```



This checks whether the current character is `*`.

- `string-ref` gets the character at index `i`
- `char=?` compares characters
- `#\*` means the character `*`

9.

```
<> racket
      (char=? (string-ref str (+ i 1)) #\/))
```



This checks whether the next character is `/`.

So together, lines 7–9 are checking:

"Are the current two characters exactly `*/`?"

10.

```
<> racket
      (helper (+ i 2) #f)
```



If we found `*/`, then:

- move forward by 2 positions, because we consumed both `*` and `/`
- set `in-comment?` to `#f`, meaning we are no longer inside a comment

So this exits comment mode.

11.

```
<> racket
(helper (+ i 1) #t))]
```

If we did **not** find `*/` , then:

- skip the current character
- move to the next one
- stay inside the comment

So everything inside the comment is ignored.

Third branch: when we are NOT inside a comment

12.

```
<> racket
[else
```

This means:

If we are not at the end, and not inside a comment, then we are reading normal program text.

13.

```
<> racket
(if (and (< (+ i 1) len)
```

Again, this makes sure we can safely look at two characters.

This time we are checking for the start of a comment.

14.

```
<> racket
(char=? (string-ref str i) #\/)
```

Checks whether current character is `/` .

15.

```
<> racket
(char=? (string-ref str (+ i 1)) #\*))
```

Checks whether next character is `*` .

So together, lines 13–15 ask:

“Do we see `/*` right here?”

16.

```
</> racket
(string-append " " (helper (+ i 2) #t))
```

If yes, then we found the start of a comment.

So:

- add a single space " " to the output
- move past both characters /*
- switch into comment mode with #t

Why insert a space?

Because comments act like separators, and we don't want tokens to accidentally stick together.

Example:

```
</> racket
"x:=5;/*comment*/y:=6;"
```

Without a space, removing the comment could risk merging nearby text.

17.

```
</> racket
(string-append
```

If it is not the start of a comment, then this is an ordinary character we want to keep.

We use `string-append` to build the result string.

18.

```
</> racket
(string (string-ref str i))
```

This gets the current character and turns it into a one-character string.

Why?

Because `string-append` works with strings, not raw characters.

Example:

If current character is `x`, this becomes:

```
</> racket
"x"
```

19.

```
<> racket
(helper (+ i 1) #f))]]))
```

Then it appends the rest of the processed string:

- move to next character
- stay outside comment mode (#f)

So this means:

"Keep this character, then continue scanning."

Final call

20.

```
<> racket
(helper 0 #f))
```

This starts the helper function.

- 0 means start at the first character
- #f means we are not inside a comment at the beginning

Big picture of the recursion

The helper is basically doing this:

- If end of string → stop
- If inside comment:
 - look for */
 - otherwise skip characters
- If outside comment:
 - look for /*
 - otherwise keep characters

Example walkthrough

Suppose input is:

```
<> racket
"x := 5; /* hi */ y := 6;"
```

Start:

- i = 0
- not in comment
- copies x , space , : , = , etc.

When it reaches / followed by * :

- adds " "
- enters comment mode

Inside comment:

- skips `h` , `i` , space
- finds `*/`
- exits comment mode

Then continues copying:

```
<> racket
" y := 6;"
```



Final result:

```
<> racket
"x := 5;   y := 6;"
```



Why this works well

This version is good because:

- it is simple
- it clearly separates comment mode vs normal mode
- it follows the assignment rule that comments are non-nested
- it preserves spacing safely

One thing to notice

This function does **not** report an error for an unclosed comment like:

```
<> racket
"x := 5; /* comment"
```



It would just keep skipping until end of file.

Depending on your assignment, you may want to later improve it so it raises an error for unterminated comments.

In one sentence

This function scans the string left to right, remembers whether it is inside a comment, skips everything inside comments, and copies everything else into a new string.

If you want, I can next explain it using a **trace table** with `i` , current character, and `in-comment?` at each step.

can you do it with car cdr

Yes — and this is actually a **very Racket-style way** to do it 🍌

Instead of indexing with `string-ref`, we convert the string into a **list of characters** and use `car` / `cdr`

🧠 Idea (car/cdr version)

We treat the string like this:

```
<> racket
"x := 5;"
```

👉 becomes:

```
<> racket
'(#\x #\space #\: #\= #\space #\5 #\;)
```

Then we recursively process:

- `car` → current character
- `cdr` → rest of the list

✅ Code using `car` / `cdr`

```
<> racket

(define (strip-comments str)
  (define chars (string->list str))

  (define (helper lst in-comment?)
    (cond
      ;; End of list
      [(null? lst) '()]

      ;; If inside a comment
      [in-comment?
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\*)
                (char=? (car (cdr lst)) #\/))
           ;; Found "*/" → exit comment
           (helper (cdr (cdr lst)) #f)
           ;; Skip character
           (helper (cdr lst) #t))])

      ;; If NOT inside comment
      [else
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\/))
           (helper (cdr (cdr lst)) #f)
           (helper (cdr lst) #t))])])

  (helper chars #f))
```

🔍 Explanation (Focused on car/cdr logic)

💠 Convert string → list

```
<> racket
(string->list str)
```

Now you can use:

- `(car lst)` → first character
- `(cdr lst)` → rest of characters

◆ Base case

```
<> racket  
[(null? lst) '()]
```



If list is empty → return empty list

◆ Checking next character

```
<> racket  
(pair? (cdr lst))
```



This ensures:

👉 there is a “next character”

(otherwise `(car (cdr lst))` would crash)

◆ Detecting `/`*

```
<> racket  
  
(char=? (car lst) #\/  
(char=? (car (cdr lst)) #\*))
```



👉 `current = /`

👉 `next = *`

◆ Detecting `*`/

```
<> racket  
  
(char=? (car lst) #\*)  
(char=? (car (cdr lst)) #\//)
```



◆ Skipping two characters

```
<> racket  
  
(cdr (cdr lst))
```



This removes both characters (like `/`* or `*/`)

◆ Building result

Keep character:

```
<> racket
(cons (car lst) ...)
```



Insert space when comment starts:

```
<> racket
(cons #\space ...)
```



◆ Final conversion

```
<> racket
(list->string ...)
```



Turns character list back into string

Example

```
<> racket
(strip-comments "x := 5; /* hi */ y := 6;")
```



👉 Output:

```
<> racket
"x := 5;   y := 6;"
```



🔥 Why this version is NICE

- Pure functional style
- Uses recursion naturally
- No indexing (cleaner logic)
- Matches how parsers are often written in Scheme/Racket

⚠ Common Mistakes

- ✗ Forgetting (pair? (cdr lst)) → crash
- ✗ Using (car (cdr lst)) without checking
- ✗ Not skipping both characters (cdr (cdr lst))

💡 Interview-Level Insight

This version is essentially doing:

👉 pattern matching on character streams

Which is EXACTLY what your parser will do later with tokens.

Next Step

If you want, I can now help you:

- Build `tokenize` using **`car/cdr` style**
- Or show how this connects directly to your parser

Just tell me 👍

explain this code line by line

Of course — here is the **`car/cdr` version** explained line by line.

```
<> racket

(define (strip-comments str)
  (define chars (string->list str))

  (define (helper lst in-comment?)
    (cond
      [(null? lst) '()]

      [in-comment?
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\*)
                (char=? (car (cdr lst)) #\/))
           (helper (cdr (cdr lst)) #f)
           (helper (cdr lst) #t))]

      [else
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\/)
                (char=? (car (cdr lst)) #\*))
           (cons #\space (helper (cdr (cdr lst)) #t))
           (cons (car lst) (helper (cdr lst) #f)))]))

  (list->string (helper chars #f)))
```

Line-by-line explanation

1.

```
<> racket

(define (strip-comments str)
```

This defines a function called `strip-comments`.

- `define` creates the function
- `strip-comments` is the function name
- `str` is the input string

So this function takes the whole source code as a string.

2.

```
<> racket
(define chars (string->list str))
```

This converts the string into a list of characters.

Example:

```
<> racket
(string->list "ab")
```

becomes:

```
<> racket
'(#\a #\b)
```

Why do this?

Because now we can process the input using:

- `car` → first character
- `cdr` → rest of the list

This is the main reason this version uses `car` and `cdr` .

3.

```
<> racket
(define (helper lst in-comment?)
```

This defines an inner recursive function called `helper` .

It takes two parameters:

- `lst` = the current list of characters still left to process
- `in-comment?` = whether we are currently inside a comment

So at every step, `helper` works on the remaining characters.

4.

```
<> racket
(cond
```

This starts a `cond` , which is Racket's multi-branch conditional.

It checks conditions one by one.

First case: no characters left

5.

```
<> racket
[(null? 1st) '()]
```

This means:

If `1st` is empty, return the empty list.

Why?

Because there are no characters left to process.

Important:

Since we are working with a **list of characters**, the result of `helper` is also a **list of characters**, not a string yet.

So:

- empty result = `'()`

Second case: currently inside a comment

6.

```
<> racket
[in-comment?
```

This branch runs when `in-comment?` is true.

That means we already saw `/*`, and now we should ignore everything until we find `*/`.

7.

```
<> racket
(if (and (pair? (cdr 1st))
```

This starts an `if` that checks whether the current and next characters form `*/`.

First part:

```
<> racket
(pair? (cdr 1st))
```

This checks whether `cdr 1st` is a non-empty list.

Why is this needed?

Because later we do:

```
<> racket
(car (cdr 1st))
```

which means "look at the next character."

If there is no next character, that would fail.

So this line is a safety check:

"Do we have at least two characters to examine?"

8.

```
<> racket
(char=? (car lst) #\*)
```

This checks whether the current character is `*`.

- `(car lst)` = first character in the list
- `#\*` = the character `*`

9.

```
<> racket
(char=? (car (cdr lst)) #\/))
```

This checks whether the next character is `/`.

- `(cdr lst)` = rest of the list
- `(car (cdr lst))` = first character of the rest = next character
- `#\/` = the character `/`

So lines 7–9 together ask:

"Do the next two characters equal `*/`?"

10.

```
<> racket
(helper (cdr (cdr lst)) #f)
```

If the answer is yes, then we found the end of the comment.

So:

- `(cdr (cdr lst))` skips both characters:
 - the `*`
 - the `/`
- `#f` means we are no longer inside a comment

So this exits comment mode and continues scanning after `*/`.

11.

```
<> racket
```

```
(helper (cdr lst) #t))]
```

If we are inside a comment but did **not** find `*/` , then:

- skip the current character by moving to `(cdr lst)`
- stay in comment mode with `#t`

So this ignores everything inside the comment.

Third case: not inside a comment

12.

```
<> racket
      [else
```



This means:

We are not at end of input, and we are not inside a comment.

So we are reading normal program text.

13.

```
<> racket
      (if (and (pair? (cdr lst))
```



Again, this is a safety check before looking at two characters.

We want to see whether the next two characters are `/*` .

So first we verify that a next character exists.

14.

```
<> racket
      (char=? (car lst) #\/) )
```



Checks whether the current character is `/` .

15.

```
<> racket
      (char=? (car (cdr lst)) #\*))
```



Checks whether the next character is `*` .

So lines 13–15 together ask:

“Do we see the start of a comment: `/*` ?”

16.

```
<> racket
      (cons #\space (helper (cdr (cdr lst)) #t))
```

If yes, we found the beginning of a comment.

So we do three things:

a. Insert a space

```
<> racket
#\space
```

This keeps tokens from accidentally running together.

b. Skip both characters

```
<> racket
      (cdr (cdr lst))
```

This removes / and * .

c. Enter comment mode

```
<> racket
#t
```

So from here on, characters will be ignored until */ .

Why cons ?

Because the result is being built as a list of characters.

cons adds one character to the front of the result list.

17.

```
<> racket
      (cons (car lst) (helper (cdr lst) #f))))))
```

If we are not at the start of a comment, then the current character is normal text and should be kept.

So:

a. (car lst)

Take the current character

b. (helper (cdr lst) #f)

Process the rest of the list

c. cons

Add the current character to the front of the processed rest

So this keeps ordinary characters unchanged.

Final line

18.

```
<> racket
(list->string (helper chars #f))
```

This starts the recursion and converts the final result back into a string.

Step 1:

```
<> racket
(helper chars #f)
```

Start with:

- all characters in `chars`
- `#f` because we are not inside a comment initially

Step 2:

```
<> racket
(list->string ...)
```

The helper returns a list of characters, such as:

```
<> racket
'(#\x #\space #\y)
```

This converts it back into a string:

```
<> racket
"x y"
```

Big picture of what the helper is doing

At each step:

- If nothing left → stop
- If inside a comment:
 - check for `*/`
 - otherwise skip characters
- If outside a comment:
 - check for `/*`
 - otherwise keep the current character

What `car` and `cdr` are doing here

This code treats the input as a stream of characters.

car

Gets the current character

Example:

```
<> racket
(car '#\a #\b #\c)
```

returns:

```
<> racket
#\a
```

cdr

Gets the rest of the characters

Example:

```
<> racket
(cdr '#\a #\b #\c)
```

returns:

```
<> racket
'\b #\c)
```

car (cdr lst)

Means:

"get the next character"

cdr (cdr lst)

Means:

"skip two characters"

That is why this works so nicely for detecting:

- /*
- */

Small example trace

Suppose input is:

```
<> racket
"x/*hi*/y"
```

After `string->list`, it becomes something like:

```
<> racket
'(#\x #\ / #\* #\h #\i #\* #\ / #\y)
```

First step:

- not in comment
- current char = x
- not /*
- keep x

Next step:

- current char = /
- next char = *
- start comment
- insert space
- enter comment mode

Inside comment:

- skip h
- skip i

Then:

- current char = *
- next char = /
- end comment
- exit comment mode

Final:

- keep y

Result becomes:

```
</> racket  
"x y"
```



Why cons is used instead of string-append

Because in this version the helper is building a **list of characters**, not a string.

So:

- cons adds one character to a character list
- then list->string converts the final list into a string

This is more natural when using car and cdr .

In one sentence

This function turns the string into a character list, recursively scans it using car and cdr , skips characters inside /* ... */ , keeps normal characters, and then converts the cleaned character list back into a string.

I can also rewrite this code with comments on every single line inside the code itself, which might be easier to study.

how to handle unterminated comments

Great question — this is exactly the kind of **edge case professors love** 🧐

🧠 What is an *unterminated comment*?

A comment starts with:

```
<> racket  
/*
```



but **never ends with**:

```
<> racket  
*/
```



Example:

```
<> racket  
"x := 5; /* this never ends..."
```



👉 Your current function will:

- enter comment mode
- reach end of input
- silently stop ❌

But you should:

👉 **raise an error**

✅ What you should do

When you reach the **end of input while still inside a comment**, throw an error like:

```
<> racket  
error: "Unterminated comment"
```



🔧 Fixing your car/cdr version

We modify the **base case**:

❌ **Current version**

```
<> racket

[(null? lst) '()]
```

This ignores whether we are inside a comment.

✓ Fixed version

```
<> racket

[(null? lst)
 (if in-comment?
     (error "Unterminated comment")
     '())]
```

✓ Full Updated Code (with error handling)

```
<> racket

(define (strip-comments str)
  (define chars (string->list str))

  (define (helper lst in-comment?)
    (cond
      ;; End of input
      [(null? lst)
       (if in-comment?
           (error "Unterminated comment")
           '())]

      ;; Inside comment
      [in-comment?
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\*)
                (char=? (car (cdr lst)) #\/))
           ;; End comment
           (helper (cdr (cdr lst)) #f)
           ;; Skip char
           (helper (cdr lst) #t))])

  ;; Outside comment
  (helper chars #f))
```

✍ Test Cases

✓ Valid comment

```
<> racket

(strip-comments "x := 5; /* ok */ y := 6;")
```

✓ Works fine

✗ Unterminated comment

```
<> racket
```

```
(strip-comments "x := 5; /* broken comment")
```

👉 Output:

```
error: Unterminated comment
```



⚠️ Why This Matters for Your Assignment

From your spec:

- Comments are **strictly defined**
- Unexpected symbols → syntax/lexical errors  CS 441 Program 2 Spr 2026

So:

👉 Unterminated comment = **invalid program**

💡 Extra (Nice Upgrade)

If you want to be *extra professional*, you can include position info:

```
<> racket
(error (format "Unterminated comment at position ~a" i))
```



But that requires tracking index (`i`) — optional unless required.

🔥 Key Insight

👉 The only place you can detect this error is:

```
<> racket
when input ends AND in-comment? = true
```



That's the **exact condition**.

🚀 Next Step

You're now ready to move to:

👉 `tokenize` (scanner) — this is where things get more complex (numbers, identifiers, operators)

If you want, I can help you build that step-by-step 🍌

Sources

yes please use token as struct

Perfect — using a `struct` for tokens is a good choice.

First: define the token struct

A simple version is:

```
<> racket

(struct token (type value) #:transparent)
```

This means each token has:

- `type` → what kind of token it is
- `value` → the actual content, when needed

Example tokens:

```
<> racket

(token 'ID "x")
(token 'INT 10)
(token 'ASSIGN ":=")
(token 'PLUS "+")
```

`#:transparent` lets you see the contents when testing.

A good tokenizer design

Your tokenizer should do this:

1. skip whitespace
2. recognize comments are already removed
3. recognize multi-character operators first
 - `:=`
 - `!=`
 - `>=`
 - `<=`

4. recognize single-character operators
 - `+ - * / = > < ( ) ;`
5. recognize identifiers and keywords
6. recognize integers and floating-point numbers
7. raise errors on invalid input

Suggested starter code

This version uses:

- `token` as a struct
- character-list processing with `car` / `cdr`
- helper functions for identifiers and numbers

```
<> racket
#lang racket

(struct token (type value) #:transparent)

(define (tokenize str)
  (define chars (string->list str))

  ;; -----
  ;; basic helpers
  ;; -----
  (define (whitespace? ch)
    (or (char-whitespace? ch)
        (char=? ch #\newline)
        (char=? ch #\tab)))

  (define (letter? ch)
    (or (char-alphabetic? ch)
        (char=? ch #\_)))

  (define (id-tail-char? ch)
    (or (char-alphabetic? ch)
        (char-numeric? ch)))
```

Example tests

Assignment

```
<> racket

(tokenize "x := 10;")
```

Output:

```
<> racket

(list
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'INT 10)
  (token 'SEMI ";"))
```

Expression

```
<> racket
(tokenize "x := 1 + 2 * 3 - 4 / 2;")
```



Keywords

```
<> racket
(tokenize "WHILE x > 0 DO PRINT x; END")
```



Floating point

```
<> racket
(tokenize "y := -10.25;")
```



Errors

```
<> racket
(tokenize "x := 05;")
```



```
<> racket
(tokenize "y := ! 5;")
```



```
<> racket
(tokenize "x : = 5;")
```



Important note about unary vs binary + and -

This starter tokenizer uses a simple rule:

- if + or - is immediately followed by a digit, it treats it as part of a number
- otherwise it becomes PLUS or MINUS

That works for many cases like:

```
<> racket
x := -5;
y := +10.25;
```



But your assignment's full rule is a bit more precise:

whether + / - is unary depends on the **previous token context**, not just the next character.

 CS 441 Program 2 Spr 2026

So later you may want to improve that by tracking the previous token type.

Best next move

Do this in order:

1. test identifiers and keywords
2. test `:=`, `!=`, `>=`, `<=`
3. test integers
4. test floating point
5. test invalid cases
6. then improve signed-number handling

The next thing I'd recommend is adding the **previous-token-aware unary/binary sign logic**, because that's usually where scanner bugs happen.

Sources

leading and tail characters?

Great question — this is a **core concept for identifiers**, and it comes directly from your grammar.

What are *leading* and *tail* characters?

From your assignment grammar:

```
<ID> ::= <leading_char> <tail_chars>
```



That means:

👉 An identifier (variable name) has:

1. **one leading character** (first character)
2. **zero or more tail characters** (rest of the name)

💠 1. Leading Character (FIRST character)

From your spec:

```
<leading_char> ::= <letter> | "_"
```



So the **first character must be**:

✅ A letter:

a b c ... z A B C ... Z

OR

✅ Underscore:

-

✔ Valid examples:

```
x
y
abc
_variable
X1
```

✖ Invalid examples:

```
1x      ; cannot start with digit
-abc    ; cannot start with -
@name   ; invalid symbol
```

◆ 2. Tail Characters (REST of the identifier)

From your spec:

```
<tail_char> ::= <letter> | <digit> | "_" | "-"
```

So after the first character, you can have:

- ✔ letters
- ✔ digits
- ✔ underscore _
- ✔ hyphen -

✔ Valid identifiers:

```
x1
var_2
data-set
_temp123
A_B-C
```

✖ Invalid:

```
x@1      ; @ not allowed
x$y      ; $ not allowed
```

🔥 Important difference

Position	Allowed
First char	letter OR _
Rest chars	letter, digit, _ , -

Why this matters in your tokenizer

When scanning:

Step 1:

If you see a **leading character** → start reading an identifier

Step 2:

Keep reading **tail characters** until something invalid appears

In your code

These functions implement that rule:

◆ Leading character check

```
<> racket

(define (letter? ch)
  (or (char-alphabetic? ch)
      (char=? ch #\_)))
```



👉 This matches <leading_char>

◆ Tail character check

```
<> racket

(define (id-tail-char? ch)
  (or (char-alphabetic? ch)
      (char-numeric? ch)
      (char=? ch #\_ )
      (char=? ch #\_-)))
```



👉 This matches <tail_char>

How identifier parsing works

Example:

```
<> racket

"data-set123;"
```



Step-by-step:

1. d → valid leading char → start identifier
2. keep reading:
 - a ✓
 - t ✓
 - a ✓
 - - ✓ (allowed in tail)
 - s ✓

- e ✓
- t ✓
- 1 ✓
- 2 ✓
- 3 ✓

3. stop at ;

👉 result:

```
<> racket
(token 'ID "data-set123")
```



⚠ Important edge case

✗ This is invalid:

```
<> racket
"1abc"
```



Why?

- 1 is NOT a valid leading character
- So this is NOT an identifier

👉 Your tokenizer should treat it as a number first, then error later if needed

💡 Summary (super simple)

- **Leading char = first character**
→ must be letter or _
- **Tail chars = rest**
→ can be letter, digit, _ , or -

🚀 If you want next

I can:

- walk through `read-identifier` step-by-step
- or help you debug your tokenizer with test cases

Just tell me 👍

explain the code for tokenizer by med sized chunkc

Absolutely — I'll explain the tokenizer in **medium-sized chunks**, so each piece is big enough to make sense but not overwhelming.

I'll use the code I gave you and break it into logical sections.

Chunk 1: Token struct and tokenizer setup

```
<> racket

#lang racket

(struct token (type value) #:transparent)

(define (tokenize str)
  (define chars (string->list str))
```

What this does

#lang racket

This tells Racket which language you are using.

```
(struct token (type value) #:transparent)
```

This creates a token structure.

Each token has 2 fields:

- **type** → what kind of token it is
- **value** → the actual text or numeric value

Examples:

```
<> racket

(token 'ID "x")
(token 'INT 10)
(token 'ASSIGN ":=")
```

Why #:transparent ?

It lets you see inside the struct when you test it.

Without it, Racket hides the contents.

With it, you can see:

```
<> racket

(token 'ID "x")
```

instead of some unreadable internal form.

```
(define (tokenize str) ... )
```

This defines the tokenizer function.

It takes one input:

- **str** = the source code as a string

```
(define chars (string->list str))
```

This converts the string into a list of characters.

Example:

```
<> racket  
"abc"
```



becomes:

```
<> racket  
'(#\a #\b #\c)
```



Why do this?

Because then you can process the input using:

- `car` → current character
- `cdr` → remaining characters

So the tokenizer works by scanning a list of chars.

Chunk 2: Character classification helpers

```
<> racket  
  
(define (whitespace? ch)  
  (or (char-whitespace? ch)  
      (char=? ch #\newline)  
      (char=? ch #\tab)))  
  
(define (letter? ch)  
  (or (char-alphabetic? ch)  
      (char=? ch #\_)))  
  
(define (id-tail-char? ch)  
  (or (char-alphabetic? ch)  
      (char-numeric? ch)  
      (char=? ch #\_)  
      (char=? ch #\_-)))  
  
(define (digit? ch)  
  (char-numeric? ch))
```



What this chunk does

These are helper functions that classify characters.

`whitespace?`

Checks whether a character is whitespace.

This includes:

- normal spaces
- newline
- tab

Tokenizer uses this to skip spaces between tokens.

`letter?`

Despite the name, this is really checking whether a character is a valid **leading character** for an identifier.

It allows:

- alphabetic letters
- underscore `_`

So it matches your grammar rule for the first character of an identifier.

Examples that return true:

- `a`
- `Z`
- `_`

Examples that return false:

- `1`
 - `-`
 - `;`
-

`id-tail-char?`

Checks whether a character is valid in the **rest** of an identifier.

It allows:

- letters
- digits
- underscore `_`
- hyphen `-`

So it matches your grammar's tail-character rule.

Example:

For identifier `"data-set1"` :

- `d` must satisfy `letter?`
 - `a t a - s e t 1` must satisfy `id-tail-char?`
-

`digit?`

Checks whether a character is numeric.

Used for reading integers and floating point numbers.

Chunk 3: `take-while`

`<> racket`



```
(define (take-while pred lst)
  (cond
    [(null? lst) (cons '() '())]
    [(pred (car lst))
     (define rest-result (take-while pred (cdr lst)))
     (cons (cons (car lst) (car rest-result))
           (cdr rest-result))]
    [else
     (cons '() lst)]))
```

What this does

This is a very useful helper.

It keeps taking characters from the front of a list **as long as** they satisfy some condition.

It returns a pair:

- the characters it collected
- the characters left over

Example

Suppose:

```
<> racket
(take-while char-numeric? '(#\1 #\2 #\3 #\a #\b))
```

It returns something like:

```
<> racket
(cons '(#\1 #\2 #\3) '(#\a #\b))
```

Meaning:

- collected digits = 123
- remaining chars = ab

Line-by-line idea

```
[(null? lst) (cons '() '())]
```

If the list is empty:

- collected chars = empty
- remaining chars = empty

```
[(pred (car lst)) ... ]
```

If the current character satisfies the condition:

- keep it
- recursively continue on the rest

```
(define rest-result (take-while pred (cdr lst)))
```

This recursively processes the rest of the list.

```
`(cons (cons (car lst) (car rest-result))
```

```
(cdr rest-result))`
```

This adds the current character to the collected part.

So if recursive call collected:

```
<> racket
'(#\2 #\3)
```

and current char is `#\1`, this builds:

```
<> racket
'(#\1 #\2 #\3)
```

```
[else (cons '() lst)]
```

If current character does not match:

- stop collecting
- return empty collected list
- leave the whole current list untouched

This is how the tokenizer knows when a token ends.

Chunk 4: Keywords vs identifiers

```
<> racket

(define (keyword-or-id s)
  (cond
    [(string=? s "IF")      (token 'IF "IF")]
    [(string=? s "THEN")   (token 'THEN "THEN")]
    [(string=? s "ELSE")   (token 'ELSE "ELSE")]
    [(string=? s "END")    (token 'END "END")]
    [(string=? s "WHILE")  (token 'WHILE "WHILE")]
    [(string=? s "DO")     (token 'DO "DO")]
    [(string=? s "PRINT")  (token 'PRINT "PRINT")]
    [else                  (token 'ID s)]))
```

What this does

When you read a word like `"IF"` or `"x"`, you need to decide:

- is it a keyword?
- or just a normal identifier?

This function does that.

Examples

```
<> racket
(keyword-or-id "IF")
```

returns:

```
<> racket
(token 'IF "IF")
```

But:

```
<> racket
(keyword-or-id "count")
```

returns:

```
<> racket
(token 'ID "count")
```

Why this is needed

The scanner reads both keywords and identifiers in basically the same way:

- start with a valid leading char
- continue through tail chars

So after it reads the whole word, it checks:

"Is this one of the reserved words?"

If yes → make a keyword token

If no → make an identifier token

Chunk 5: Reading an identifier

```
<> racket

(define (read-identifier lst)
  (define result (take-while id-tail-char? lst))
  (define lexeme (list->string (car result)))
  (cons (keyword-or-id lexeme) (cdr result)))
```

What this does

This function reads an identifier or keyword starting from the current position.

It assumes the first character is already a valid start for an identifier.

Step-by-step

```
(define result (take-while id-tail-char? lst))
```

This collects all consecutive identifier-tail characters.

Example input:

```
<> racket
'(#\x #\1 #\- #\y #\; ...)
```

It collects:

```
<> racket
'(#\x #\1 #\- #\y)
```

and leaves:

```
<> racket
'(#\; ...)
```

```
(define lexeme (list->string (car result)))
```

The collected characters are turned into a string.

Example:

```
<> racket
'(#\x #\1 #\- #\y)
```

becomes:

```
<> racket
"x1-y"
```

```
(cons (keyword-or-id lexeme) (cdr result))
```

This returns:

- the token for that word
- the remaining characters after it

So if lexeme is "WHILE" , token becomes:

```
<> racket
(token 'WHILE "WHILE")
```

If lexeme is "counter-1" , token becomes:

```
<> racket
(token 'ID "counter-1")
```

Chunk 6: Reading numbers

```
<> racket
```

```

(define (read-number lst)
  (define int-part-result (take-while digit? lst))
  (define int-chars (car int-part-result))
  (define rest1 (cdr int-part-result))

  (define int-str (list->string int-chars))

  (cond
    [(and (pair? rest1)
          (char=? (car rest1) #\.))
     (define rest-after-dot (cdr rest1))
     (unless (and (pair? rest-after-dot) (digit? (car rest-after-dot)))
       (error "Lexical Error: floating point must have digits after decimal point"))
     (define frac-result (take-while digit? rest-after-dot))
     (define frac-chars (car frac-result))
     (define rest2 (cdr frac-result))
     (define full-str
       (string-append int-str "." (list->string frac-chars)))
     (cons (token 'FP (string->number full-str)) rest2)]
    [else
     (when (and (> (string-length int-str) 1)

```

What this does

This reads either:

- an integer
- or a floating-point number

It starts at a digit.

First part: read the digits before the decimal

```
(take-while digit? lst)
```

This grabs all consecutive digits from the front.

Example:

For input:

```
<> racket
'(#\1 #\2 #\. #\5 ...)
```

it collects:

```
<> racket
'(#\1 #\2)
```

and leaves:

```
<> racket
'(#\. #\5 ...)
```

int-str

Turns those digits into a string like "12" .

Floating-point case

```
[(and (pair? rest1) (char=? (car rest1) #\.) ... )]
```

If the next character is a dot, treat it as a floating point number.

rest-after-dot

Moves past the decimal point.

unless ...

Checks that there is at least one digit after the dot.

This prevents invalid numbers like:

- 12.
- 5.

frac-result

Reads all digits after the decimal point.

Example:

if input is 12.345 , it collects 345 .

full-str

Builds the full number string:

```
<> racket
"12.345"
```



```
(token 'FP (string->number full-str))
```

Creates a floating-point token.

Example:

```
<> racket
(token 'FP 12.345)
```



Integer case

If there is no decimal point, it is treated as an integer.

Leading zero check

```
<> racket
```



```
(when (and (> (string-length int-str) 1)
          (char=? (string-ref int-str 0) #\0))
      (error ...))
```

This rejects integers like:

- 05
- 007

but allows:

- 0

Final result

Creates:

```
<> racket
(token 'INT 123)
```



and returns remaining characters.

Chunk 7: Signed numbers

```
<> racket

(define (read-signed-number lst)
  (define sign-char (car lst))
  (define rest (cdr lst))
  (if (and (pair? rest) (digit? (car rest)))
      (let* ([result (read-number rest)]
              [tok (car result)]
              [remaining (cdr result)]
              [val (token-value tok)]
              [signed-val (if (char=? sign-char #\-) (- val) val)]
              [new-type (token-type tok)])
        (cons (token new-type signed-val) remaining))
      #f))
```



What this does

This tries to read a number with a sign at the front:

- +5
- -10
- +3.14

Step-by-step

sign-char

Stores the sign:

- +
- or -

rest

Moves to the characters after the sign.

```
if (and (pair? rest) (digit? (car rest)))
```

Only treat the sign as part of a number if it is followed by a digit.

So:

- `-5` → signed number
- `+10.2` → signed number
- `x - y` → minus operator, not signed number

```
read-number rest
```

Reads the unsigned number part.

Example:

For `-12.5`, it reads `12.5`.

```
token-value tok
```

Gets the numeric value from the token.

Example:

If token is:

```
<> racket
(token 'FP 12.5)
```



then value is `12.5`.

```
signed-val
```

If sign is `-`, negate the value.

If sign is `+`, keep it.

So:

- `-12.5` becomes `-12.5`
- `+12.5` becomes `12.5`

Return value

Returns a token with the same type (`INT` or `FP`) but signed value.

If it is not really a signed number, returns `#f`.

That tells the scanner:

"this plus/minus should probably be treated as an operator instead."

Chunk 8: Main scanner skeleton

```
<> racket

(define (scan lst tokens)
  (cond
    [(null? lst) (reverse tokens)]
```

What this does

This is the main recursive scanning function.

It takes:

- `lst` = remaining characters
- `tokens` = tokens collected so far

Why `tokens` is built backwards

Every time a new token is found, it is added using `cons`, which puts it at the front.

That is efficient.

So at the end, we use:

```
<> racket

(reverse tokens)
```

to restore correct order.

Base case

If no characters remain:

- return all collected tokens in the right order

Chunk 9: Skipping spaces and handling multi-character operators

```
<> racket

[(whitespace? (car lst))
 (scan (cdr lst) tokens)]

[(and (pair? (cdr lst))
      (char=? (car lst) #\:)
      (char=? (car (cdr lst)) #\=))
 (scan (cdr (cdr lst))
      (cons (token 'ASSIGN "!=") tokens))]

[(and (pair? (cdr lst))
      (char=? (car lst) #\!)
      (char=? (car (cdr lst)) #\=))
 (scan (cdr (cdr lst))
      (cons (token 'NEQ "!=") tokens))]
```

What this does

Whitespace case

If current char is whitespace:

- ignore it
- move on

Whitespace is not a token.

`:=`

Checks whether current two characters are `:=` .

If yes:

- make assignment token
 - skip both characters
-

`!=`

Checks whether current two characters are `!=` .

If yes:

- make not-equal token
 - skip both characters
-

Why multi-character operators are checked first

Because if you checked `:` or `!` by themselves first, you might incorrectly split:

- `:=`
- `!=`

So longest/special combinations are checked before single-character operators.

Chunk 10: Remaining comparisons, identifiers, numbers, and signs

`<>` racket



```

[(char=? (car lst) #\!)
 (error "Lexical Error: ! is not a valid token unless followed by =")]

[(and (pair? (cdr lst))
      (char=? (car lst) #\>)
      (char=? (car (cdr lst)) #\=))
 (scan (cdr (cdr lst))
       (cons (token 'GE ">=") tokens)))]

[(and (pair? (cdr lst))
      (char=? (car lst) #\<)
      (char=? (car (cdr lst)) #\=))
 (scan (cdr (cdr lst))
       (cons (token 'LE "<=") tokens)))]

[(letter? (car lst))
 (define result (read-identifier lst))
 (scan (cdr result) (cons (car result) tokens))]

[(digit? (car lst))
 (define result (read-number lst))
 (scan (cdr result) (cons (car result) tokens))]

```

What this does

Bare !

If scanner sees ! not followed by =, it throws a lexical error.

This matches your assignment.

>= and <=

These are scanned like := and !=:

- look ahead two characters
- if matched, create one token
- skip both

Identifier / keyword

If current character is a valid leading identifier char:

- call `read-identifier`
- get token plus remaining input
- continue scanning

Number

If current character is a digit:

- call `read-number`
- get INT or FP token
- continue scanning

+ and -

These are tricky because they can mean two things:

- sign of a number: `-5`

- binary operator: $x - y$

So scanner tries `read-signed-number` first.

If that works:

- produce signed numeric token

If not:

- produce `PLUS` or `MINUS` operator token

This is a simplified rule and may need improvement later with `previous-token` context.

Chunk 11: Single-character tokens and final errors

⟨⟩ racket

```
[(char=? (car lst) #\*)
 (scan (cdr lst) (cons (token 'MULT "*") tokens)))]
[(char=? (car lst) #\/)
 (scan (cdr lst) (cons (token 'DIV "/") tokens)))]
[(char=? (car lst) #\=)
 (scan (cdr lst) (cons (token 'EQ "=") tokens)))]
[(char=? (car lst) #\>)
 (scan (cdr lst) (cons (token 'GT ">") tokens)))]
[(char=? (car lst) #\<)
 (scan (cdr lst) (cons (token 'LT "<") tokens)))]
[(char=? (car lst) #\()
 (scan (cdr lst) (cons (token 'LPAREN "(") tokens)))]
[(char=? (car lst) #\))
 (scan (cdr lst) (cons (token 'RPAREN ")") tokens)))]
[(char=? (car lst) #\;)
 (scan (cdr lst) (cons (token 'SEMI ";" ) tokens)))]

[(char=? (car lst) #\: )
 (error "Lexical Error: expected :=, but found bare :")]

[else
 (error (format "Lexical Error: unexpected character ~a" (car lst)))]
```

What this does

These cases handle all the simple one-character tokens:

- `*`
- `/`
- `=`
- `>`
- `<`
- `(`
- `)`
- `;`

For each one:

- create token
- move to rest of characters
- continue scanning

Bare `:`

If the scanner sees `:` by itself, it raises an error.

Why?

Because your grammar only allows `:=`, not plain `:`.

So:

- `:=` is valid
- `:` alone is invalid

Final else

If no case matches, it means current character is something illegal, like:

- `@`
- `$`
- strange punctuation

So scanner raises a lexical error.

Last line

```
</> racket
(scan chars '())
```



This starts scanning:

- with the full character list
- and an empty token list

Big picture summary

The tokenizer works like this:

1. convert source string into a character list
2. scan left to right
3. skip whitespace
4. detect operators and delimiters
5. read identifiers / keywords
6. read numbers
7. raise errors for invalid input
8. return list of token structs

One important note

Sources

The tokenizer I gave is a **strong starter**, but the `+` / `-` handling may need refinement because your

```

- - - - -

```

So this version is good for understanding structure, but later you may want to improve the sign logic.

If you want, next I can explain just modify the code for unary vs binary depends on the previous token context, not only the next character.

`-5;`

Yes — that's the right improvement.

Your assignment says `+` or `-` is unary **only when the previous token context allows a new expression to start**. If the previous token is an ID, number, or `)`, then `+` / `-` should be treated as **binary operators**.

 CS 441 Program 2 Spr 2026

So we should stop deciding based only on the **next character**, and also track the **previous token**.

Main idea

Treat `+` / `-` as unary when there is no completed value before it

Unary is allowed after things like:

- start of input
- `:=`
- `(`
- comparison operators
- `THEN`
- `DO`
- `PRINT`
- another operator like `+` `-` `*` `/`

Binary is required after things like:

- ID
- INT
- FP
- `)`

Modified tokenizer

This version changes `scan` so it keeps a third argument:

```
<> racket
prev-token-type
```



Then it uses that to decide whether `+` / `-` should be unary or binary.

```
<> racket
#lang racket

(struct token (type value) #:transparent)

(define (tokenize str)
  (define chars (string->list str))

  ;; -----
  ;; character helpers
  ;; -----
  (define (whitespace? ch)
    (or (char-whitespace? ch)
        (char=? ch #\newline)
        (char=? ch #\tab)))

  (define (leading-char? ch)
    (or (char-alphabetic? ch)
        (char=? ch #\_)))
```



```
(define (id-tail-char? ch)
  (or (char-alphabetic? ch)
```

What changed

1. scan now tracks previous token type

Before:

```
<> racket
(scan 1st tokens)
```

Now:

```
<> racket
(scan 1st tokens prev-type)
```

So the scanner remembers what came before.

2. Added this helper

```
<> racket
(define (value-ending-token? prev-type)
  (member prev-type '(ID INT FP RPAREN)))
```

These are the token types that mean:

"an expression/value just ended."

So after these, + or - should be binary.

Examples:

- x - 5
- 10 + 2
- (a+b) - 3

3. Unary context is the opposite

```
<> racket
(define (unary-context? prev-type)
  (not (value-ending-token? prev-type)))
```

So unary is allowed after:

- start of input
- :=
- (
- comparison operators
- THEN, DO, PRINT, etc.

4. read-signed-number now checks context too

Before it only checked:

- is next character a digit?

Now it checks:

- is unary allowed here?
- is next character a digit?

So `-5` is only read as a signed number if the previous token context allows unary.

How it behaves now

Unary minus

```
<> racket  
  
(tokenize "x := -5;")
```

Result:

```
<> racket  
  
(list  
  (token 'ID "x")  
  (token 'ASSIGN ":=")  
  (token 'INT -5)  
  (token 'SEMI ";"))
```

Because after `ASSIGN`, unary is allowed.

Binary minus

```
<> racket  
  
(tokenize "x := y - 5;")
```

Result:

```
<> racket  
  
(list  
  (token 'ID "x")  
  (token 'ASSIGN ":=")  
  (token 'ID "y")  
  (token 'MINUS "-")  
  (token 'INT 5)  
  (token 'SEMI ";"))
```

Because after `ID`, minus must be binary.

Unary plus

```
<> racket  
  
(tokenize "y := +10.25;")
```

Result:

```
<> racket

(list
  (token 'ID "y")
  (token 'ASSIGN ":=")
  (token 'FP 10.25)
  (token 'SEMI ";"))
```



Binary plus

```
<> racket

(tokenize "PRINT x + 1;")
```



Result:

```
<> racket

(list
  (token 'PRINT "PRINT")
  (token 'ID "x")
  (token 'PLUS "+")
  (token 'INT 1)
  (token 'SEMI ";"))
```



Because after ID, + is binary.

One limitation still left

This version handles:

- -5
- +10.25

But it does **not** treat this as a unary expression:

```
<> racket

x := -(y + 1);
```



because your grammar defines unary sign only as part of <Number>, not as a general unary operator before any factor. So this behavior is consistent with the grammar you were given.

 CS 441 Program 2 Spr 2026

Best next test cases

Run these:

```
<> racket

(tokenize "x := -5;")
(tokenize "x := y - 5;")
(tokenize "y := +10.25;")
(tokenize "PRINT x * y + 1;")
(tokenize "WHILE x > 0 DO x := x - 1; END")
```



If you want, I can next rewrite this with **comments on the modified lines only**, so it's easier to compare old version.

Sources

now give the full code for tokenize and use index based scanning not list

Absolutely — here is a **full** tokenize **implementation** using:

- `struct token`
- **index-based scanning**
- unary vs binary `+` `/` `-` based on **previous token context**
- keyword recognition
- identifier rules
- integer / floating-point rules
- lexical error handling

This is aligned with the assignment's scanner requirements.  CS 441 Program 2 Spr 2026

Full code

```
<> racket
#lang racket

(struct token (type value) #:transparent)

(define (tokenize str)
  (define len (string-length str))

  ;; -----
  ;; character helpers
  ;; -----
  (define (at-end? i)
    (>= i len))

  (define (current-char i)
    (string-ref str i))

  (define (has-next? i)
    (< (+ i 1) len))

  (define (next-char i)
    (string-ref str (+ i 1)))
```



What this handles correctly

Identifiers and keywords

```
<> racket

(tokenize "WHILE x DO PRINT y; END")
```



Assignment

```
<> racket
(tokenize "x := 10;")
```



Integer and floating-point numbers

```
<> racket
(tokenize "x := 5;")
(tokenize "y := 10.25;")
```



Unary vs binary

```
<> racket
(tokenize "x := -5;")
(tokenize "x := y - 5;")
(tokenize "y := +10.25;")
```



Relational operators

```
<> racket
=  !=  >  >=  <  <=
```



Important note

This tokenizer assumes you already ran `strip-comments` first, which matches the two-stage design in the assignment: comments are removed before tokenizing.  [CS 441 Program 2 Spr 2026](#)

So normally you would do:

```
<> racket
(tokenize (strip-comments source-string))
```



Example outputs

Example 1

```
<> racket
(tokenize "x := 10;")
```



Output:

```
<> racket
(list
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'INT 10)
  (token 'SEMI ";"))
```



Example 2

```
<> racket  
  
(tokenize "x := y - 5;")
```



Output:

```
<> racket  
  
(list  
  (token 'ID "x")  
  (token 'ASSIGN ":=")  
  (token 'ID "y")  
  (token 'MINUS "-")  
  (token 'INT 5)  
  (token 'SEMI ";"))
```



Example 3

```
<> racket  
  
(tokenize "y := +10.25;")
```



Output:

```
<> racket  
  
(list  
  (token 'ID "y")  
  (token 'ASSIGN ":=")  
  (token 'FP 10.25)  
  (token 'SEMI ";"))
```



Error cases it catches

Leading zero integer

```
<> racket  
  
(tokenize "x := 05;")
```



Invalid float

```
<> racket  
  
(tokenize "x := 12.;")  
(tokenize "x := 1.2.3;")
```



Invalid !

```
<> racket  
  
(tokenize "y := 10 ! 5;")
```



Bare colon

```
<> racket
```



```
(tokenize "x : = 5;")
```

One limitation to know

This follows your grammar's idea that unary sign belongs to a **number**, not a general factor. So it supports:

```
<> racket
-5
+10.25
```



but not something like:

```
<> racket
-(x + 1)
```



which is consistent with the grammar shown in the assignment. CS 441 Program 2 Spr 2026

If you want, next I can give you a **full test suite** for this tokenizer with valid and invalid inputs.

Sources

now before giving me the code tell me what to do for parser what functions to write

For the parser, think in terms of **one function per grammar rule**. The assignment explicitly wants a recursive-descent parser where grammar functions are mutually recursive and use helpers like `peek` and `consume`. CS 441 Program 2 Spr 2026

Big picture

Your parser should take:

```
<> racket
(listof token)
```



and return:

```
<> racket
(cons ast remaining-tokens)
```



for most parsing functions.

The top-level one should eventually make sure **nothing is left over** except end-of-input.

CS 441 Program 2 Spr 2026

Functions you should write

1. Parser state helpers first

Write these before the grammar functions.

`peek`

Looks at the current token without removing it.

`consume`

Checks that the current token is the expected type.

- if correct → return remaining tokens
- if wrong → raise syntax error

optional: `token-matches?`

A small helper that checks token type.

optional: `expect-and-return`

Sometimes useful if you want to consume and also use the token's value.

2. Top-level parser function

`parse-program`

This is the entry point.

It should:

- call `parse-stmt-list`
- wrap the result as `(program ...)` if that is the AST format you want
- verify all tokens are consumed

So this is the function you call after tokenizing.

3. Statement-list layer

`parse-stmt-list`

This parses one or more statements.

Grammar:

```
<StmtList> ::= <Statement> | <Statement> <StmtList>
```



So it should:

- parse a statement
- then keep parsing more statements while the next token still begins a statement

A statement can start with:

- ID
- IF
- WHILE
- PRINT

It should stop when it reaches tokens that mean the block is over, like:

- ELSE
- END
- or end of token list

This function is very important because it handles blocks inside IF and WHILE .

4. Statement dispatcher

parse-statement

This should inspect the next token and decide which specific statement parser to call.

If next token is:

- ID → parse-assign-stmt
- IF → parse-if-stmt
- WHILE → parse-while-stmt
- PRINT → parse-print-stmt

If none match, raise syntax error.  CS 441 Program 2 Spr 2026

Specific statement functions

5. parse-assign-stmt

Grammar:

```
<AssignStmt> ::= <ID> "!=" <Expression> ";"
```



It should:

- read identifier
- require ASSIGN
- parse an expression
- require SEMI

AST idea:

```
<> racket
(assign x expr)
```



6. parse-if-stmt

Grammar:

```
<IfStmt> ::= "IF" <Comparison> "THEN" <StmtList> "ELSE" <StmtList> "END"
```



It should:

- consume IF
- parse comparison
- consume THEN
- parse then-block with parse-stmt-list
- consume ELSE
- parse else-block with parse-stmt-list

- consume END

AST idea:

```
<> racket
(if condition then-block else-block)
```

You can decide whether `then-block` and `else-block` are lists of statements or wrapped in something like `(block ...)`.

7. parse-while-stmt

Grammar:

```
<WhileStmt> ::= "WHILE" <Comparison> "DO" <StmtList> "END"
```

It should:

- consume WHILE
- parse comparison
- consume DO
- parse body using `parse-stmt-list`
- consume END

AST idea:

```
<> racket
(while condition body)
```

8. parse-print-stmt

Grammar:

```
<PrintStmt> ::= "PRINT" <Expression> ";"
```

It should:

- consume PRINT
- parse expression
- consume SEMI

AST idea:

```
<> racket
(print expr)
```

Comparison and expression layer

9. parse-comparison

Grammar:

```
<Comparison> ::= <Expression> <RelOp> <Expression>
```

It should:

- parse left expression
- parse relational operator
- parse right expression

AST ideas:

```
<> racket
(eq left right)
(ne left right)
(gt left right)
(ge left right)
(lt left right)
(le left right)
```



This function is used inside `IF` and `WHILE`. CS 441 Program 2 Spr 2026

10. parse-relop

This reads one relational operator token and turns it into the AST operator name.

Example mapping:

- `EQ` → `'eq` or `'=` depending on your chosen AST style
- `NEQ` → `'neq`
- `GT` → `'gt`
- `GE` → `'ge`
- `LT` → `'lt`
- `LE` → `'le`

It can return:

```
<> racket
(cons op remaining-tokens)
```



or just return the symbol and let caller manage tokens.

Arithmetic precedence functions

These are the classic recursive-descent precedence levels from the assignment. CS 441 Program 2 Spr 2026

11. parse-expression

Grammar:

```
<Expression> ::= <Term> { ("+" | "-") <Term> }
```



This handles:

- addition
- subtraction

It should:

- parse first term
- then loop while next token is `PLUS` or `MINUS`
- each time parse another term and build a bigger AST

This gives **left associativity**.

Example:

```
10 - 5 - 2
```



becomes:

```
<> racket
(- (- 10 5) 2)
```



12. parse-term

Grammar:

```
<Term> ::= <Factor> { ("*" | "/" ) <Factor> }
```



This handles:

- multiplication
- division

It should:

- parse first factor
- then loop while next token is `MULT` or `DIV`
- build left-associated AST

This is how multiplication/division bind tighter than addition/subtraction.

13. parse-factor

Grammar:

```
<Factor> ::= <ID> | <Number> | "(" <Expression> ")"
```



This is the base level.

It should handle:

- identifiers
- integers
- floating-point numbers
- parenthesized expressions

If it sees `LPAREN`, it should:

- consume `(`
- call `parse-expression`
- require `RPAREN`

This is the “recursive leap” the assignment talks about.  CS 441 Program 2 Spr 2026

Optional small helpers

These are not required, but they make the parser cleaner.

statement-start?

Returns true if a token can begin a statement:

- ID
- IF
- WHILE
- PRINT

Very useful inside `parse-stmt-list` .

relop-token?

Returns true if token is a relational operator.

addop-token?

Checks for `PLUS` or `MINUS` .

mulop-token?

Checks for `MULT` or `DIV` .

Suggested build order

Write them in this order:

Stage 1: helpers

- `peek`
- `consume`
- `maybe statement-start?`

Stage 2: lowest-level expression parsing

- `parse-factor`
- `parse-term`
- `parse-expression`

Stage 3: comparisons

- `parse-relop`
- `parse-comparison`

Stage 4: statements

- `parse-assign-stmt`
- `parse-print-stmt`
- `parse-if-stmt`
- `parse-while-stmt`
- `parse-statement`

Stage 5: block and top level

- `parse-stmt-list`
- `parse-program`

That order works because each layer depends on the simpler layers below it.

What each grammar function should return

A very clean pattern is:

```
<> racket
(cons ast remaining-tokens)
```

Examples:

```
<> racket
(parse-factor tokens)
=> (cons 10 rest)
```

```
<> racket
(parse-expression tokens)
=> (cons '(+ 1 (* 2 3)) rest)
```

```
<> racket
(parse-assign-stmt tokens)
=> (cons '(assign x 10) rest)
```

Then `parse-program` can extract the final AST and verify no extra tokens remain.

AST shape you should decide early

Before coding, decide your AST naming style and stay consistent.

For example:

```
<> racket
(program stmt1 stmt2 ...)
(assign x expr)
(print expr)
(if cond then-part else-part)
(while cond body)
(+ left right)
(- left right)
(* left right)
(/ left right)
(gt left right)
```

Or use words like:

```
<> racket
(plus left right)
(minus left right)
(times left right)
(div left right)
```

Either is fine as long as you are consistent.

Most important parser idea

Your parser is basically a chain of responsibility:

- `parse-program`
 - `parse-stmt-list`

- `parse-statement`
 - one of statement parsers
 - expressions/comparisons underneath
 - `parse-expression`
 - `parse-term`
 - `parse-factor`

That layered design is what gives you precedence and correct structure.  CS 441 Program 2 Spr 2026

Minimal function list

If you want the shortest checklist, write these:

```
<> racket

peek
consume
parse-program
parse-stmt-list
parse-statement
parse-assign-stmt
parse-if-stmt
parse-while-stmt
parse-print-stmt
parse-comparison
parse-relop
parse-expression
parse-term
parse-factor
```



Next, I can lay out the exact skeleton code for those parser functions without filling in all the logic yet.

Sources

give me the code

Here's a full **recursive-descent parser** for the grammar in your assignment, assuming your scanner already produces `token` structs with fields `(type value)`. It includes the helper functions, all parser functions, precedence handling, comparisons, and statement/block parsing. This matches the grammar structure the assignment describes for `parse-program`, `parse-stmt-list`, `parse-statement`, `parse-expression`, `parse-term`, and `parse-factor`.  CS 441 Program 2 Spr 2026

 CS 441 Program 2 Spr 2026

```

<> racket

#lang racket

;; Assumes your scanner uses this token struct:
(struct token (type value) #:transparent)

;; =====
;; Helpers
;; =====

(define (peek tokens)
  (if (null? tokens)
      'EOF
      (car tokens)))

(define (token-type-safe tok)
  (if (eq? tok 'EOF)
      'EOF
      (token-type tok)))

(define (token-value-safe tok)
  (if (eq? tok 'EOF)
      'EOF

```

How to use it

If your tokenizer returns a token list, call:

```

<> racket

(parse tokens)

```

Example:

```

<> racket

(define toks
  (list
    (token 'ID "x")
    (token 'ASSIGN ":=")
    (token 'INT 10)
    (token 'SEMI ";")
    (token 'WHILE "WHILE")
    (token 'ID "x")
    (token 'GT ">")
    (token 'INT 0)
    (token 'DO "DO")
    (token 'ID "x")
    (token 'ASSIGN ":=")
    (token 'ID "x")
    (token 'MINUS "-")
    (token 'INT 1)
    (token 'SEMI ";")
    (token 'END "END")))

(parse toks)

```

Output:

```

<> racket

'(program
  (assign x 10)
  (while (gt x 0)
    ((assign x (- x 1)))))

```

Small note

In this version:

- `if` stores then/else parts as statement lists
- `while` stores body as a statement list

That is fine structurally, but if you want, I can next adjust the AST shape so it looks **exactly** like the samp style in your assignment, with bodies flattened a bit more. The assignment's parser architecture and precedence requirements are what this code is following.  CS 441 Program 2 Spr 2026

 CS 441 Program 2 Spr 2026

Sources

```
peek
consume
parse-program
parse-stmt-list
parse-statement
parse-assign-stmt
parse-if-stmt
parse-while-stmt
parse-print-stmt
parse-comparison
parse-relop
parse-expression
parse-term
parse-factor explain the order the funtions are called
```

Great question — this is the **key to understanding recursive descent parsing**.

I'll explain it in a way that shows **actual flow (who calls whom)**, not just a list.

Big Picture

Think of the parser like a **chain of responsibility from top → bottom**:

```
parse-program
  ↓
parse-stmt-list
  ↓
parse-statement
  ↓
(statement-specific function)
  ↓
(parse-comparison OR parse-expression)
  ↓
parse-expression
  ↓
parse-term
  ↓
parse-factor
```

👉 Each layer handles a different level of grammar

🧭 Full Call Order (Hierarchy)

↑ TOP Top Level

```
parse-program
  → parse-stmt-list
```

📦 Statement Level

```
parse-stmt-list
  → parse-statement
    → parse-assign-stmt
    → parse-if-stmt
    → parse-while-stmt
    → parse-print-stmt
```

🧠 Expression / Condition Level

```
parse-if / parse-while
  → parse-comparison
  → parse-expression
```

🧮 Arithmetic Level

```
parse-expression
  → parse-term
  → parse-factor
```

🔄 Let's Walk Through a REAL Example

Input:

```
x := 1 + 2 * 3;
```



Step-by-step call flow

1. Start

```
parse-program
```



calls:

```
parse-stmt-list
```



2. First statement

```
parse-stmt-list  
  → parse-statement
```



Looks at token:

```
ID → assignment
```



So:

```
parse-statement  
  → parse-assign-stmt
```



3. Assignment parsing

```
parse-assign-stmt  
  → consume ID  
  → consume :=  
  → parse-expression
```



4. Expression parsing begins

```
parse-expression  
  → parse-term
```



5. Term parsing

```
parse-term  
  → parse-factor
```



6. Factor

```
parse-factor  
  → reads 1
```



returns:

```
1
```



Back to parse-expression

Now sees + , so:

```
parse-expression  
  → parse-term (again)
```



7. Second term

```
parse-term  
  → parse-factor → reads 2
```



Then sees * , so:

```
parse-term  
  → parse-factor → reads 3
```



Builds:

```
(* 2 3)
```



Back to parse-expression

Now combine:

```
(+ 1 (* 2 3))
```



Back to assign

```
(assign x (+ 1 (* 2 3)))
```



Back to stmt-list

No more statements → done

Back to program

Wrap everything:

```
(program
  (assign x (+ 1 (* 2 3))))
```



IF / WHILE Example Flow

Input:

```
WHILE x > 0 DO x := x - 1; END
```



Call chain

```
parse-program
  → parse-stmt-list
    → parse-statement
      → parse-while-stmt
```



Inside while

```
parse-while-stmt
  → parse-comparison
```



Comparison

```
parse-comparison
  → parse-expression (left)
  → parse-relop
  → parse-expression (right)
```



Then body

```
parse-while-stmt
  → parse-stmt-list
    → parse-statement
      → parse-assign-stmt
        → parse-expression
```



Key Insight

There are 3 main layers

Program / Statements layer

- parse-program
- parse-stmt-list
- parse-statement

👉 decides structure of program

2 Control / logic layer

- parse-if-stmt
- parse-while-stmt
- parse-comparison
- parse-relop

👉 handles conditions and control flow

3 Arithmetic layer

- parse-expression
- parse-term
- parse-factor

👉 handles precedence and math

🔄 Why this order matters

Because of **precedence**

```
factor (highest)
term (* /)
expression (+ -)
comparison (>, <, etc.)
statement
program
```



Each level calls the level **below** it

🧠 Mental Model (IMPORTANT)

Think:

👉 Top-down parser = "what am I expecting?"

- Program expects statements
 - Statement expects specific pattern
 - Expression expects terms
 - Term expects factors
-

🔄 Call Stack Visualization

For:

```
x := 1 + 2 * 3;
```



Call stack looks like:

```
parse-program
  parse-stmt-list
    parse-statement
      parse-assign-stmt
        parse-expression
          parse-term
            parse-factor
```

Then unwinds upward building AST.

 Summary Table

Function	Calls
parse-program	parse-stmt-list
parse-stmt-list	parse-statement
parse-statement	specific stmt
parse-assign-stmt	parse-expression
parse-if/while	parse-comparison + stmt-list
parse-comparison	parse-expression
parse-expression	parse-term
parse-term	parse-factor
parse-factor	base case

 One-Line Summary

👉 The parser starts from **program** → **statements** → **expressions** → **factors**, and builds the AST **from bottom back up**.

If you want next, I can:

- draw a **visual tree diagram** based on the code you gave me for parser give an example and show
- or trace a full IF example step by step how does it build the ast tree

Both are super helpful for exams 🙌

Yes — let's trace it using the parser code I gave you and show **exactly how the AST gets built step by step**.

I'll use this example because it shows precedence clearly:

```
x := 1 + 2 * 3;
```

According to the assignment, multiplication/division must bind tighter than addition/subtraction, so this should become an AST like:

```
<> racket

(program
  (assign x (+ 1 (* 2 3))))
```

That matches the precedence rules in the spec.

 CS 441 Program 2 Spr 2026

 CS 441 Program 2 Spr 2026

1. Token list going into the parser

Assume the tokenizer already produced:

```
<> racket

(list
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'INT 1)
  (token 'PLUS "+")
  (token 'INT 2)
  (token 'MULT "*")
  (token 'INT 3)
  (token 'SEMI ";"))
```

Now we call:

```
<> racket

(parse tokens)
```

which calls:

```
<> racket

(parse-program tokens)
```

2. parse-program

Code idea:

```
<> racket

(define result (parse-stmt-list tokens))
(define stmts (car result))
(define remaining (cdr result))
(cons (cons 'program stmts) remaining)
```

So `parse-program` does **not** build the whole tree by itself.

It first says:

“Go parse the statement list for me.”

So the call becomes:

```
<> racket

(parse-stmt-list tokens)
```

3. parse-stmt-list

This function parses one statement first:

```
<> racket
(define first-result (parse-statement tokens))
```

So now we call:

```
<> racket
(parse-statement tokens)
```

At this moment, the next token is:

```
<> racket
(token 'ID "x")
```

So `parse-statement` decides this must be an assignment.

4. parse-statement

It looks at the next token type:

- ID → call `parse-assign-stmt`
- IF → call `parse-if-stmt`
- WHILE → call `parse-while-stmt`
- PRINT → call `parse-print-stmt`

Since the first token is `ID`, it does:

```
<> racket
(parse-assign-stmt tokens)
```

5. parse-assign-stmt

This function follows the grammar:

```
<ID> ":" "=" <Expression> ";"
```

So it processes the input in pieces.

Step 5a: read the variable name

The first token is:

```
<> racket
(token 'ID "x")
```

The code converts that name into a symbol:

```
<> racket
(string->symbol (token-value id-tok))
```

So:

```
<> racket
"x" → 'x
```

Now it has the variable name:

```
<> racket
x
```

Not the whole AST yet — just the variable name.

Step 5b: consume ID

It removes the first token:

```
<> racket
(consume tokens 'ID)
```

Remaining tokens now are:

```
<> racket
(list
  (token 'ASSIGN ":=")
  (token 'INT 1)
  (token 'PLUS "+")
  (token 'INT 2)
  (token 'MULT "*")
  (token 'INT 3)
  (token 'SEMI ";"))
```

Step 5c: consume ASSIGN

It removes := :

```
<> racket
(consume tokens1 'ASSIGN)
```

Remaining tokens now are:

```
<> racket
(list
  (token 'INT 1)
  (token 'PLUS "+")
  (token 'INT 2)
  (token 'MULT "*")
  (token 'INT 3)
  (token 'SEMI ";"))
```

Now it must parse the expression on the right side:

```
<> racket
```

```
(parse-expression tokens2)
```

This is where the AST starts getting built from the arithmetic structure.

6. parse-expression

Grammar:

```
<Expression> ::= <Term> { ("+" | "-") <Term> }
```

So `parse-expression` first calls:

```
<> racket
(parse-term tokens)
```

It does that because **terms are higher precedence than expressions**.

Current tokens:

```
<> racket
(INT 1) (PLUS "+") (INT 2) (MULT "*") (INT 3) (SEMI ";")
```

7. First parse-term

Grammar:

```
<Term> ::= <Factor> { ("*" | "/") <Factor> }
```

So it first calls:

```
<> racket
(parse-factor tokens)
```

8. First parse-factor

Grammar:

```
<Factor> ::= <ID> | <Number> | "(" <Expression> ")"
```

The next token is:

```
<> racket
(token 'INT 1)
```

So `parse-factor` returns:

```
<> racket
(cons 1 remaining-tokens)
```

That means:

- AST built so far = 1
- remaining tokens =

```
<> racket

(list
  (token 'PLUS "+")
  (token 'INT 2)
  (token 'MULT "*")
  (token 'INT 3)
  (token 'SEMI ";"))
```

So the first little AST node created is just:

```
<> racket

1
```

9. Back to first parse-term

Now `parse-term` asks:

Is the next token `MULT` or `DIV` ?

But the next token is:

```
<> racket

PLUS
```

So no — term is done.

It returns:

```
<> racket

(cons 1 remaining-tokens)
```

So now `parse-expression` receives:

- left AST = 1
- remaining tokens start with `PLUS`

10. Back to parse-expression

Now `parse-expression` asks:

Is the next token `PLUS` or `MINUS` ?

Yes — it is `PLUS` .

So now it knows it is building something like:

```
<> racket

(+ left right)
```

At this moment:

- left side is already known: 1

- right side still needs to be parsed by another `parse-term`

So it consumes the `PLUS` and calls:

```
<> racket
(parse-term ...)
```

on the remaining tokens:

```
<> racket
(INT 2) (MULT "*" ) (INT 3) (SEMI ";" )
```

11. Second `parse-term`

Again, `parse-term` starts by calling:

```
<> racket
(parse-factor ...)
```

12. Second `parse-factor`

The next token is:

```
<> racket
(token 'INT 2)
```

So it returns:

```
<> racket
(cons 2 remaining-tokens)
```

Remaining tokens now:

```
<> racket
(list
 (token 'MULT "*" )
 (token 'INT 3)
 (token 'SEMI ";" ))
```

So current AST inside this term is:

```
<> racket
2
```

13. Back to second `parse-term`

Now `parse-term` checks:

Is next token `MULT` or `DIV` ?

Yes — it sees `MULT` .

So now it knows it must build:

```
<> racket
(* left right)
```

Currently:

- `left = 2`

It consumes `MULT` and calls `parse-factor` again on:

```
<> racket
(INT 3) (SEMI ";" )
```

14. Third `parse-factor`

The next token is:

```
<> racket
(token 'INT 3)
```

So it returns:

```
<> racket
(cons 3 remaining-tokens)
```

Remaining tokens now:

```
<> racket
(list
 (token 'SEMI ";" ))
```

So right side is:

```
<> racket
3
```

15. Back to second `parse-term` — build subtree

Now `parse-term` has:

- `left = 2`
- `operator = *`
- `right = 3`

So it builds this AST subtree:

```
<> racket
(* 2 3)
```

This is the first real operator node built in the tree.

So second `parse-term` returns:

```
<> racket
(cons '(* 2 3) remaining-tokens)
```

Remaining tokens:

```
<> racket
(SEMI)
```

16. Back to `parse-expression` — build bigger subtree

Now `parse-expression` has:

- `left = 1`
- `operator = +`
- `right = (* 2 3)`

So it builds:

```
<> racket
(+ 1 (* 2 3))
```

This is the full right-hand-side expression AST.

Then it checks again:

Is the next token another `PLUS` or `MINUS` ?

No — the next token is `SEMI` .

So expression is done.

It returns:

```
<> racket
(cons '(+ 1 (* 2 3)) remaining-tokens)
```

Remaining tokens:

```
<> racket
(SEMI)
```

17. Back to `parse-assign-stmt` — build assignment node

Now `parse-assign-stmt` has:

- `variable name = x`
- `parsed expression = (+ 1 (* 2 3))`

So it builds:

```
<> racket
(assign x (+ 1 (* 2 3)))
```

Then it consumes the semicolon ; .

So it returns:

```
<> racket
(cons '(assign x (+ 1 (* 2 3))) '())
```

That means:

- statement AST is finished
- no tokens remain

18. Back to parse-stmt-list

parse-stmt-list got its first statement:

```
<> racket
(assign x (+ 1 (* 2 3)))
```

Then it checks whether there is another statement after that.

There are no tokens left, so it stops and returns the statement list:

```
<> racket
(cons
  (list '(assign x (+ 1 (* 2 3))))
  '())
```

So the statement list is:

```
<> racket
((assign x (+ 1 (* 2 3))))
```

Notice that this is a **list of statements**.

19. Back to parse-program — wrap with program

Finally, parse-program takes that statement list and wraps it:

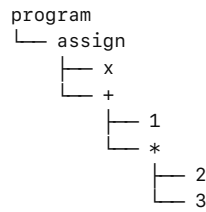
```
<> racket
(program (assign x (+ 1 (* 2 3))))
```

So final AST is:

```
<> racket
'(program
  (assign x (+ 1 (* 2 3))))
```

Tree shape visualization

Here is the tree that got built:



That is why multiplication is deeper in the tree than addition.

The exact order nodes were built

This is important:

Built first:

```
<> racket
1
2
3
```

These came from `parse-factor`.

Then built:

```
<> racket
(* 2 3)
```

This came from `parse-term`.

Then built:

```
<> racket
(+ 1 (* 2 3))
```

This came from `parse-expression`.

Then built:

```
<> racket
(assign x (+ 1 (* 2 3)))
```

This came from `parse-assign-stmt`.

Then built:

```
<> racket
(program (assign x (+ 1 (* 2 3))))
```

This came from `parse-program`.

So the parser **calls functions from top down**, but the AST is mostly **built from bottom up**.

Why precedence works

Because:

- `parse-expression` does not directly grab numbers
- it first asks `parse-term`
- `parse-term` first asks `parse-factor`

So when parsing `1 + 2 * 3`:

- expression gets left side `1`
- on the right side, it asks for a full term
- that term consumes `2 * 3` before expression gets a chance to combine with `+`

That is exactly how the precedence hierarchy in the assignment is enforced.  CS 441 Program 2 Spr 2026

Short version of the flow

For this input:

```
x := 1 + 2 * 3;
```

the calls go like this:

```
parse-program
→ parse-stmt-list
→ parse-statement
→ parse-assign-stmt
→ parse-expression
→ parse-term
→ parse-factor    => 1
→ parse-term
→ parse-factor    => 2
→ parse-factor    => 3
```

and the AST gets assembled in this order:

```
1
2
3
(* 2 3)
(+ 1 (* 2 3))
(assign x (+ 1 (* 2 3)))
(program (assign x (+ 1 (* 2 3))))
```

If you want, I can do the same kind of step-by-step trace for an `IF` or `WHILE` example next.

Sources

yes pls

Let's do a **WHILE** example step by step, because it shows both:

- how a **comparison** is parsed
- how a **statement list inside a block** is parsed

Example input

```
WHILE x > 0 DO
  x := x - 1;
END
```



Assume the tokenizer gives this token list:

```
<> racket

(list
  (token 'WHILE "WHILE")
  (token 'ID "x")
  (token 'GT ">")
  (token 'INT 0)
  (token 'DO "DO")
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'ID "x")
  (token 'MINUS "-")
  (token 'INT 1)
  (token 'SEMI ";")
  (token 'END "END"))
```



The grammar says a while statement looks like:

```
WHILE <Comparison> DO <StmtList> END
```



which is exactly what your parser code is following. CS 441 Program 2 Spr 2026

Final AST we expect

Using the parser code I gave, the result will look like:

```
<> racket

'(program
  (while (gt x 0)
    ((assign x (- x 1)))))
```



Notice the body is a **list of statements**, so the assignment appears inside an extra pair of parentheses.

Step 1: parse-program

We call:

```
<> racket
```



```
(parse tokens)
```

which calls:

```
<> racket
(parse-program tokens)
```

parse-program does this:

1. calls parse-stmt-list
2. gets back a list of statements
3. wraps them with program

So first real parsing call is:

```
<> racket
(parse-stmt-list tokens)
```

Step 2: parse-stmt-list

This parses the first statement by calling:

```
<> racket
(parse-statement tokens)
```

The next token is:

```
<> racket
(token 'WHILE "WHILE")
```

So parse-statement chooses:

```
<> racket
(parse-while-stmt tokens)
```

Step 3: parse-while-stmt

This function follows:

```
WHILE <Comparison> DO <StmtList> END
```

So it processes the pieces in this order:

1. consume WHILE
2. parse comparison
3. consume DO
4. parse statement list for the loop body
5. consume END

Step 3a: consume WHILE

It removes the first token.

Remaining tokens now:

```
</> racket

(list
  (token 'ID "x")
  (token 'GT ">")
  (token 'INT 0)
  (token 'DO "DO")
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'ID "x")
  (token 'MINUS "-")
  (token 'INT 1)
  (token 'SEMI ";")
  (token 'END "END"))
```

Now it calls:

```
</> racket

(parse-comparison tokens1)
```

Step 4: parse-comparison

Grammar:

```
<Comparison> ::= <Expression> <RelOp> <Expression>
```

So it does:

1. parse left expression
2. parse relational operator
3. parse right expression

Current tokens start with:

```
</> racket

ID x
```

Step 4a: parse left expression

Call:

```
</> racket

(parse-expression ...)
```

Step 5: left parse-expression

Grammar:

```
<Expression> ::= <Term> { ("+" | "-") <Term> }
```

So it starts with:

```
<> racket
(parse-term ...)
```

Step 6: left parse-term

Grammar:

```
<Term> ::= <Factor> { ("*" | "/" ) <Factor> }
```

So it starts with:

```
<> racket
(parse-factor ...)
```

Step 7: left parse-factor

Next token is:

```
<> racket
(token 'ID "x")
```

So parse-factor returns:

```
<> racket
(cons 'x remaining-tokens)
```

Remaining tokens now:

```
<> racket
(list
 (token 'GT ">")
 (token 'INT 0)
 (token 'DO "DO")
 (token 'ID "x")
 (token 'ASSIGN ":=")
 (token 'ID "x")
 (token 'MINUS "-")
 (token 'INT 1)
 (token 'SEMI ";")
 (token 'END "END"))
```

So the left expression AST is just:

```
<> racket
x
```

Since next token is GT , not * , / , + , or - , both parse-term and parse-expression finish.

So left side of comparison is:

```
<> racket
```

```
x
```



Step 4b: parse relational operator

Now `parse-comparison` calls:

```
<> racket
```

```
(parse-relop ...)
```



Next token is:

```
<> racket
```

```
(token 'GT ">")
```



So `parse-relop` returns:

```
<> racket
```

```
(cons 'gt remaining-tokens)
```



Remaining tokens now:

```
<> racket
```

```
(list
  (token 'INT 0)
  (token 'DO "DO")
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'ID "x")
  (token 'MINUS "-")
  (token 'INT 1)
  (token 'SEMI ";")
  (token 'END "END"))
```



So operator is:

```
<> racket
```

```
gt
```



Step 4c: parse right expression

Now `parse-comparison` calls:

```
<> racket
```

```
(parse-expression ...)
```



Next token is:

```
<> racket
```

```
(token 'INT 0)
```



That goes through:

- `parse-expression`
- `parse-term`
- `parse-factor`

and returns:

```
<> racket
0
```

Remaining tokens now:

```
<> racket
(list
  (token 'DO "DO")
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'ID "x")
  (token 'MINUS "-")
  (token 'INT 1)
  (token 'SEMI ";")
  (token 'END "END"))
```

Step 4d: build comparison subtree

Now `parse-comparison` has:

- `left = x`
- `relop = gt`
- `right = 0`

So it builds:

```
<> racket
(gt x 0)
```

This is the first full subtree built for the while statement.

It returns:

```
<> racket
(cons '(gt x 0) remaining-tokens)
```

Step 8: back to `parse-while-stmt`

Now `parse-while-stmt` has condition:

```
<> racket
(gt x 0)
```

Next token must be `DO`.

So it consumes:

```
<> racket
(token 'DO "DO")
```



Remaining tokens now:

```
<> racket
(list
  (token 'ID "x")
  (token 'ASSIGN ":=")
  (token 'ID "x")
  (token 'MINUS "-")
  (token 'INT 1)
  (token 'SEMI ";")
  (token 'END "END"))
```



Now it must parse the loop body:

```
<> racket
(parse-stmt-list ...)
```



Step 9: parse while-body statement list

Current token is:

```
<> racket
(token 'ID "x")
```



So `parse-stmt-list` calls:

```
<> racket
(parse-statement ...)
```



which chooses:

```
<> racket
(parse-assign-stmt ...)
```



Step 10: parse-assign-stmt inside the while

This parses:

```
x := x - 1;
```



Step 10a: read variable name

First token:

```
<> racket
```



```
(token 'ID "x")
```

Converted to symbol:

```
<> racket  
x
```

Then consume ID , consume ASSIGN .

Remaining tokens now:

```
<> racket  
  
(list  
  (token 'ID "x")  
  (token 'MINUS "-")  
  (token 'INT 1)  
  (token 'SEMI ";")  
  (token 'END "END"))
```

Now parse the right-hand-side expression:

```
<> racket  
  
(parse-expression ...)
```

Step 11: parse expression $x - 1$

Current tokens:

```
<> racket  
  
ID x, MINUS, INT 1, SEMI, END
```

Step 11a: first term

parse-expression calls parse-term , which calls parse-factor .

parse-factor sees:

```
<> racket  
  
(token 'ID "x")
```

so it returns:

```
<> racket  
  
x
```

Remaining tokens:

```
<> racket  
  
MINUS, INT 1, SEMI, END
```

Step 11b: expression sees minus

Now `parse-expression` sees `MINUS`, so it knows it is building:

```
<> racket
(- left right)
```

Currently:

- `left = x`

It consumes `MINUS` and calls `parse-term` on the rest.

Step 11c: second term

`parse-term` calls `parse-factor`.

`parse-factor` sees:

```
<> racket
(token 'INT 1)
```

so it returns:

```
<> racket
1
```

Remaining tokens:

```
<> racket
SEMI, END
```

No `*` or `/`, so term finishes.

Step 11d: build arithmetic subtree

Now `parse-expression` has:

- `left = x`
- `operator = -`
- `right = 1`

So it builds:

```
<> racket
(- x 1)
```

This is the right-hand-side expression AST.

It returns:

```
<> racket
(cons '(- x 1) remaining-tokens)
```

Step 12: finish assignment inside while

Now `parse-assign-stmt` has:

- `variable = x`
- `expression = (- x 1)`

So it builds:

```
<> racket
(assign x (- x 1))
```

Then it consumes `SEMI`.

Remaining tokens now:

```
<> racket
(list
  (token 'END "END"))
```

So `parse-assign-stmt` returns:

```
<> racket
(cons '(assign x (- x 1)) remaining-tokens)
```

Step 13: finish loop body statement list

`parse-stmt-list` now has one statement in the body:

```
<> racket
(assign x (- x 1))
```

Then it checks whether another statement begins.

Next token is:

```
<> racket
END
```

That is **not** a statement starter, so body statement list stops here.

It returns the list of statements:

```
<> racket
((assign x (- x 1)))
```

Notice again: the body is a **list containing one statement**.

Step 14: finish `parse-while-stmt`

Now `parse-while-stmt` has:

- `condition = (gt x 0)`
- `body = ((assign x (- x 1)))`

Then it consumes `END` .

No tokens remain.

Now it builds the while node:

```
<> racket
(while (gt x 0) ((assign x (- x 1))))
```

That is the full AST for the while statement.

It returns:

```
<> racket
(cons '(while (gt x 0) ((assign x (- x 1)))) '())
```

Step 15: finish `parse-stmt-list` at top level

The top-level `parse-stmt-list` now has one statement:

```
<> racket
(while (gt x 0) ((assign x (- x 1))))
```

No more tokens remain, so it returns:

```
<> racket
((while (gt x 0) ((assign x (- x 1)))))
```

Step 16: finish `parse-program`

Finally `parse-program` wraps the statement list with `program` :

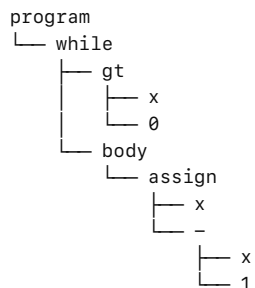
```
<> racket
(program
  (while (gt x 0)
    ((assign x (- x 1)))))
```

So final AST is:

```
<> racket
'(program
  (while (gt x 0)
    ((assign x (- x 1)))))
```

Tree visualization

Here is the AST shape:



Exact order pieces were built

This is the actual build order from smallest pieces upward:

First built:

```
<> racket

x
0
x
1
```

These are the leaves from `parse-factor`.

Then built:

```
<> racket

(gt x 0)
(- x 1)
```

These come from:

- `parse-comparison`
- `parse-expression`

Then built:

```
<> racket

(assign x (- x 1))
```

This comes from `parse-assign-stmt`.

Then built:

```
<> racket

(while (gt x 0) ((assign x (- x 1))))
```

This comes from `parse-while-stmt`.

Then built:

```
<> racket

(program (while (gt x 0) ((assign x (- x 1)))))
```

This comes from `parse-program` .

So again:

- function calls go **top-down**
- AST nodes get built **bottom-up**

Short call-flow view

For this input:

```
WHILE x > 0 DO x := x - 1; END
```

the calls go like this:

```
parse-program
→ parse-stmt-list
→ parse-statement
→ parse-while-stmt
  → parse-comparison
    → parse-expression
      → parse-term
        → parse-factor => x
      → parse-relop      => gt
    → parse-expression
      → parse-term
        → parse-factor => 0
  → parse-stmt-list
    → parse-statement
    → parse-assign-stmt
      → parse-expression
        → parse-term
          → parse-factor => x
      → parse-term
        → parse-factor => 1
```

Why `parse-stmt-list` is important here

Inside `WHILE` , the body is not just one statement by grammar — it is a **statement list**. That means this would also work:

```
WHILE x > 0 DO
  PRINT x;
  x := x - 1;
END
```

because `parse-while-stmt` calls `parse-stmt-list` , not `parse-statement` .

That follows the grammar in your assignment.  CS 441 Program 2 Spr 2026

If you want, I can do the same full trace for an `IF ... THEN ... ELSE ... END` example next.

[Sources](#)

```
/* Syntax error at EOF -- missing END for outermost IF */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * 6 * -1 + 13.0 - 0 * 0.0 + 3 * 6/3 *
12/27;
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  IF Y < 5 THEN
    WHILE Y > 0 DO
      Y := Y - 01;
      IF Y = 0. THEN
        PRINT 0;
      END; /* END IF CLAUSE */
    END; /* END WHILE */
  END; /* END OUTER IF */
ELSE
  X := -10;
  PRINT X; for this file it shows lexical error due to leading zero is
that correct?
```

Yes — getting a lexical error for 01 is correct.

In your file, this line appears inside the `WHILE` body:

```
Y := Y - 01;
```



According to the assignment's lexical rules, an `INT` cannot have a leading zero unless the number is exactly `0`. So `05` and `-07` are invalid, and by the same rule `01` is also invalid. The spec explicitly lists leading-zero integers as **lexical errors**, not syntax errors. [CS 441 Program 2 Spr 2026](#)

So your parser/scanner should report the **lexical error first**, before it ever gets to the missing `END` at EOF

Why the EOF syntax error is not reported first

Because the front end works in this order:

1. strip comments
2. tokenize / scan
3. parse

If scanning fails on `01`, tokenization stops there. The parser never gets a complete token stream, so it never gets far enough to complain about the missing outer `END`. That matches the assignment's two-stage front-end design. [CS 441 Program 2 Spr 2026](#)

There is actually more than one error in that file

Besides `01`, I can see at least these issues:

1. `Y := Y - 01;`

This is a **lexical error** because `01` has a leading zero. [CS 441 Program 2 Spr 2026](#)

2. `IF Y = 0. THEN`

This is also a **lexical error** because `0.` is not a valid floating-point number. The spec says an FP must have at least one digit before **and after** the decimal point, so `12.` and `0.` are invalid.

[CS 441 Program 2 Spr 2026](#)

3. Missing final `END`

Your comment says the outermost `IF` is missing its `END`, and that would indeed be a **syntax error at EOF** if the scanner got past the lexical errors first. The grammar for `IF` requires `IF <Comparison> THE <StmtList> ELSE <StmtList> END.` [CS 441 Program 2 Spr 2026](#)

So what should happen?

For the file exactly as written, the correct behavior is:

- first error encountered: **lexical error** at `01`
- after fixing that, next likely error: **lexical error** at `0.`
- after fixing both, then the parser should eventually report the **missing END at EOF**

In short

Yes, your scanner is right to complain about `01` first. The missing `END` is real too, but it comes later.

Sources

fix this input file/* Lexical error on line 5 */

```

X := 10;
Y := 2;
IF X > 10 THEN // The '//' is not a comment symbol.
  WHILE Y > 0 DO
    Y := Y - 1;
    IF Y = 0 THEN
      PRINT 0;
    ELSE
      Z := 0; /* SYNTAX REQUIRES ELSE CLAUSE */
    END; /* END IF CLAUSE */
  END; /* END WHILE */
ELSE
  X := 10;
END; /* END OUTER IF */

```

Here's a corrected version of the input file.

```

/* Fixed version */

X := 10;
Y := 2;
IF X > 10 THEN
  WHILE Y > 0 DO
    Y := Y - 1;
    IF Y = 0 THEN
      PRINT 0;
    ELSE
      Z := 0;
    END
  END
ELSE
  X := 10;
END

```



What I fixed

// The '//' is not a comment symbol. was removed because your language only supports /* ... */ comments.

I also removed the semicolons after END , because your grammar says:

- assignment ends with ;
- print ends with ;
- IF ... THEN ... ELSE ... END does **not** end with ;
- WHILE ... DO ... END does **not** end with ; CS 441 Program 2 Spr 2026

Why the original had problems

This line was invalid in your language:

```
IF X > 10 THEN // The '//' is not a comment symbol.
```



because // is not recognized as a comment delimiter. Only /* ... */ is valid. CS 441 Program 2 Spr 2026

Also these were not grammar-correct:

```
END;
```



for the IF and WHILE , because the grammar requires just END , not END; .  CS 441 Program 2 Spr 2026

If you want, I can also give you the exact token list and AST for this corrected file.

Sources

if i've ; after END is that incorrect?

Short answer: **Yes — in your assignment grammar, END; is incorrect.**

Why?

Look at your grammar:

```
<IfStmt> ::= "IF" <Comparison> "THEN" <StmtList> "ELSE" <StmtList> "END"  
<WhileStmt> ::= "WHILE" <Comparison> "DO" <StmtList> "END"
```



 Notice:

- END appears **by itself**
- there is **no ; after END**

But for other statements:

```
<AssignStmt> ::= <ID> "=" <Expression> ";"  
<PrintStmt> ::= "PRINT" <Expression> ";"
```



 Only **assignment** and **print** use ;  CS 441 Program 2 Spr 2026

So what happens if you write END; ?

Your parser expects:

```
... END
```



But it sees:

```
END ;
```



So:

1. parse-if-stmt or parse-while-stmt consumes END
2. Now the next token is SEMI
3. The parser goes back to parse-stmt-list
4. It expects a **new statement start**
5. But ; is **not a valid statement start**

 Result: **Syntax error**

 Example **Incorrect:**

```
WHILE x > 0 DO
  x := x - 1;
END;
```



Parser flow:

- consumes END
- sees ;
- tries to parse a statement
- fails 

 Correct:

```
WHILE x > 0 DO
  x := x - 1;
END
```

 **Important distinction**

Statement	Ends with ; ?
Assignment	 Yes
Print	 Yes
IF	 No
WHILE	 No

 Why the grammar is designed this way

Because:

- IF and WHILE are **block statements**
- They already have a clear end marker (END)
- Adding ; would be redundant

 If you wanted to allow END;

You would need to change the grammar to:

```
<IfStmt> ::= ... "END" ";"
<WhileStmt> ::= ... "END" ";"
```



and update your parser to consume SEMI after END .

 But your assignment **does NOT define it this way**, so don't do this unless explicitly told.

Final answer

👉 Yes, `END;` is incorrect in your assignment

👉 You must use just:

END



If you want, I can show you exactly where your parser throws the error when it sees `END;` 🙌

Sources

`/* Lexical error on line 4 */`

```

_X-2345ASDadlkj := 10;
3Y+43 := 2;
IF _X-2345ASDadlkj > 10 THEN
  WHILE Y > 0 DO
    Y := Y - 1;
    IF Y = 0 THEN
      PRINT 0;
    END; /* END IF CLAUSE */
  END; /* END WHILE */
ELSE
  X := 10;
  PRINT X;
END; /* END OUTER IF */ now fix this

```

Here's a corrected version that fits your assignment grammar.

```

/* Fixed version */

_X-2345ASDadlkj := 10;
Y := 2;
IF _X-2345ASDadlkj > 10 THEN
  WHILE Y > 0 DO
    Y := Y - 1;
    IF Y = 0 THEN
      PRINT 0;
    ELSE
      PRINT Y;
    END
  END
ELSE
  X := 10;
  PRINT X;
END

```



What I fixed:

`3Y+43 := 2;` was invalid because an identifier cannot start with a digit. The first character must be a letter or `_`, so I changed it to `Y := 2;`.  CS 441 Program 2 Spr 2026

I added an `ELSE` branch to the inner `IF` because your grammar requires every `IF` to have both `THEN`

I added an ELSE branch to the inner IF, because your grammar requires every IF to have both THEN and ELSE parts. IF ... THEN ... END is not valid in this assignment.  CS 441 Program 2 Spr 2026

I removed the semicolons after END, because in your grammar END closes IF and WHILE by itself. Or assignment and PRINT end with semicolons.  CS 441 Program 2 Spr 2026

One important note: I used PRINT Y; in the inner ELSE just to make the file grammatically valid. Any valid statement list there would work.

Sources

```
/* Invalid number format on line 11. Also on 12. */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * 6 * -1 + 13.0 - 0 * 0.0 + 3 * 6 / 3 * 12 / 27;
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  WHILE Y > 0 DO
    Y := Y - 01;
    IF Y = 0. THEN
      PRINT 0;
    END; /* END IF CLAUSE */
  END; /* END WHILE */
ELSE
  X := 10;
  PRINT X;
END; /* END OUTER IF */ s
now thi
```

Here's a corrected version of that input file.

```
/* Fixed version */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * 6 * -1 + 13.0 - 0 * 0.0 + 3 * 6 / 3 * 12 / 27;
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  WHILE Y > 0 DO
    Y := Y - 1;
    IF Y = 0 THEN
      PRINT 0;
    ELSE
      PRINT Y;
    END
  END
ELSE
  X := 10;
  PRINT X;
END
```

What I fixed:

`Y := Y - 01;` became `Y := Y - 1;` because `01` is an invalid integer. Your grammar allows `0` by itself, or a nonzero digit followed by more digits, so leading-zero integers are lexical errors.

 CS 441 Program 2 Spr 2026

`IF Y = 0. THEN` became `IF Y = 0 THEN` because `0.` is an invalid floating-point number. A valid FP must have at least one digit before and after the decimal point.  CS 441 Program 2 Spr 2026

I added an `ELSE` branch to the inner `IF`, because your grammar requires every `IF` to be:

`IF <Comparison> THEN <StmtList> ELSE <StmtList> END` 

so an `IF` without `ELSE` is a syntax error.  CS 441 Program 2 Spr 2026

I removed the semicolons after `END`, because in your assignment grammar `END` closes `IF` and `WHILE` by itself; only assignment and `PRINT` end with semicolons.  CS 441 Program 2 Spr 2026

I also removed the stray `s` at the end, since that would be extra invalid input after the program.

If you want, I can turn this corrected file into the exact token list your scanner should produce.

Sources

```
/* Give the nested-control logic a workout. This should parse. */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * ((6 * -1) + 13.0) - 0 * 0.0 + 3 * 6 / 3
* (12 / 27);
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  IF Y < 5 THEN
    WHILE Y > 0 DO
      Y := Y - 01;
      IF Y = 0. THEN
        PRINT 0;
      END; /* END IF CLAUSE */
    END; /* END WHILE */
  END; /* END OUTER IF */
ELSE
  X := -10;
  PRINT X;
END; /* END OUTERMOST IF */
```

This does **not** parse as written.

It has several problems:

`Y := Y - 01;` is invalid because `01` has a leading zero.

`IF Y = 0. THEN` is invalid because `0.` is not a valid floating-point literal.

`IF Y = 0 THEN ... END` is not valid in your grammar, because every `IF` needs an `ELSE` branch.

`END;` is also wrong for your grammar if you are following the assignment literally; it should be just `END`.

Here is a corrected version that should parse under the grammar you showed earlier:

```
/* Give the nested-control logic a workout. This should parse. */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * ((6 * -1) + 13.0) - 0 * 0.0 + 3 * 6 / 3 * (12 / 3)
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  IF Y < 5 THEN
    WHILE Y > 0 DO
      Y := Y - 1;
      IF Y = 0 THEN
        PRINT 0;
      ELSE
        PRINT Y;
      END
    END
  ELSE
    PRINT Y;
  END
END
```

What I changed:

- `01` → `1`
- `0.` → `0`
- added the missing `ELSE` for the inner `IF Y = 0 THEN`
- added the missing `ELSE` for the middle `IF Y < 5 THEN`
- removed semicolons after each `END`

If you want, I can also show the AST this corrected program would build.

```

/* Syntax error at EOF -- missing END for outermost IF */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * 6 * -1 + 13.0 - 0 * 0.0 + 3 * 6/3 *
12/27;
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  IF Y < 5 THEN
    WHILE Y > 0 DO
      Y := Y - 01;
      IF Y = 0. THEN
        PRINT 0;
      END; /* END IF CLAUSE */
    END; /* END WHILE */
  END; /* END OUTER IF */
ELSE
  X := -10;
  PRINT X;

```

As written, this will **not** reach the “missing END at EOF” error first.

It hits **lexical errors earlier**:

```
Y := Y - 01;
```

01 is an invalid integer because leading zeros are not allowed except for 0 by itself. So this is a lexical error.

```
IF Y = 0. THEN
```

0. is also invalid, because a floating-point number must have at least one digit **after** the decimal point. So this is another lexical error.

There are also grammar issues later:

- the inner IF Y = 0. THEN ... END is missing an ELSE
- END; is not valid if you are following the grammar literally, since IF and WHILE end with END, not END;
- and yes, the **outermost IF is missing its final END**, which would eventually cause a syntax error at EOF

So the actual error order is:

1. lexical error at 01
2. lexical error at 0.
3. syntax issues with IF / END
4. finally, missing outermost END at EOF

If your scanner stops on the first bad token, then reporting the 01 lexical error first is correct.

Here is a corrected version that preserves the same general structure and also includes the missing final

END :

```
/* Fixed version */

_X-2345ASDadlkj := 10;

/* Math check */
Y-z := _X-2345ASDadlkj + 2 - 3 * 6 * -1 + 13.0 - 0 * 0.0 + 3 * 6 / 3 * 12 / 27;
X := _X-2345ASDadlkj;
Y := Y-z;
IF X > 10 THEN
  IF Y < 5 THEN
    WHILE Y > 0 DO
      Y := Y - 1;
      IF Y = 0 THEN
        PRINT 0;
      ELSE
        PRINT Y;
      END
    END
  ELSE
    PRINT Y;
  END
END
```

So the comment `/* Syntax error at EOF -- missing END for outermost IF */` is **not the first error for this exact file**. The file has earlier lexical errors that prevent the parser from even getting that f;